NSCT Preparation
File 04: Databases (10%)

Mapped directly to the official P@SHA/HEC syllabus. Features a mix of scenario-based, conceptual, and code-tracing MCQs. Each question shows the correct answer in **bold**, followed by the explanation immediately under it.

Master Table of Contents (File 04 - Target: 217 MCQs)

#	Official Subtopic	Coverage	MCQs
01	Introduction to Database Systems	Basic	10
02	Database System Architecture	Basic	10
03	Data Models	Basic	10
04	Relational Database Concepts	Standard	15
05	Relational Algebra & Calculus	Basic	12
06	Structured Query Language (SQL)	Extended	20
07	Advanced SQL	Extended	20
08	Database Design & Normalization	Extended	20
09	Transaction Management	Standard	15
10	Concurrency Control	Standard	15
11	Recovery Management	Basic	10
12	Indexing & File Organization	Standard	15
13	Query Processing & Optimization	Basic	10
14	Database Security	Basic	10
15	Distributed Databases	Basic	10

16	NoSQL & Modern Databases	Basic	10
17	Data Warehousing & Data Mining (Intro)	Basic	15
Total MCQs (File 04)			**222**

Subtopic 01: Introduction to Database Systems (10 MCQs)

Quick Notes

- **DBMS vs File System:** DBMS solves data redundancy, data isolation, and concurrent access anomalies present in traditional OS file systems.
- **Program-Data Independence:** In a DBMS, the structure of the data files is stored in the DBMS catalog separately from the access programs.
- **Database Catalog (Metadata):** "Data about data." Stores table definitions, data types, constraints, and user access roles.
- **Schema vs Instance:** Schema is the structural design (rarely changes). Instance is the snapshot of actual data at a given moment (changes constantly).

Common Traps & Confusions

- **Database vs DBMS:** A "Database" is the actual collection of data. A "DBMS" (Database Management System) is the software (like MySQL, Oracle) used to interact with the database.
- **Redundancy vs Inconsistency:** Redundancy (storing data multiple times) is the *cause*. Inconsistency (different copies having different values) is the *resulting error*.

MCQs

Q1. (Scenario-Based) A university registrar uses a traditional File Processing System. Student addresses are stored in both a `Billing.dat` file and a `Library.dat` file. When a student moves, the address is only updated in the `Billing.dat` file. What specific database problem does this illustrate?

A) Concurrency Exception

B) Data Inconsistency resulting from Data Redundancy

C) Atomicity Failure

D) Program-Data Dependence

> Because the same data is stored in two places (Redundancy), updating one without the other leads to conflicting facts about the student's address. A DBMS prevents this via centralized storage and normalization.

Q2. (Comparison) Which of the following best describes the difference between a Database **Schema** and a Database **Instance**?

A) A schema is stored on a physical disk, while an instance resides in RAM.

B) A schema defines the application logic, while an instance defines network connections.

C) A schema is the static structural blueprint, while an instance is the dynamic set of data present at a specific moment.

D) A schema is used in relational databases, while instances are used exclusively in NoSQL.

> The schema is the defined structure (tables, columns, data types) and rarely changes. An instance is the snapshot of the data (the rows) inside those tables, which changes with every INSERT, UPDATE, or DELETE operation.

Q3. (Code-Tracing) A database administrator executes the following SQL query to inspect the database. What specific DBMS component is being accessed here?

mysql-terminal

```
SELECT table_name, engine, table_rows
FROM information_schema.tables
WHERE table_schema = 'production_db';
```

A) The transaction log buffer

B) The application data instance

C) The system catalog (metadata)

D) The external view schema

> The `information_schema` holds metadata ("data about data"). By querying it, the DBA is inspecting the structure and statistics of the database itself, not the actual business data (like customer names or orders).

Q4. (Conceptual) In traditional file systems, if you change the length of a 'ZipCode' field from 5 to 9 characters in the storage file, you must also rewrite the application code reading that file. A DBMS eliminates this requirement. This characteristic is known as:

A) ACID Compliance

B) Program-Data Independence

C) Data Isolation

D) Concurrency Control

> Program-Data Independence means the structure of the data is managed by the DBMS and separated from the application programs. Applications interact with the DBMS through APIs, shielding them from low-level file structure changes.

Q5. (Scenario-Based) Two travel agents attempt to book the last remaining seat on a flight at the exact same millisecond. In a traditional file system, both might succeed, causing an overbooking. What inherent feature of a DBMS prevents this?

A) Data Abstraction

B) Data Dictionary

C) Concurrency Control

D) Crash Recovery

> Concurrency Control (often implemented via locks) ensures that when multiple transactions attempt to update the same data simultaneously, they are executed in a safe, serializable manner, preventing lost updates or overbooking anomalies.

Q6. (Conceptual) While a DBMS offers many advantages, there are scenarios where utilizing a DBMS is considered unnecessary or harmful. Which of the following is a valid reason to avoid using a full-scale DBMS?

A) The application has strict real-time hardware requirements where DBMS software overhead is unacceptable.

B) The application requires data to be accessed by multiple users simultaneously.

C) The data is highly interrelated and requires complex querying.

D) The organization needs robust crash recovery mechanisms.

> A DBMS introduces overhead (memory, CPU processing, disk I/O for logging). For simple, embedded, or strict real-time systems (like a pacemaker or a simple sensor logger), the overhead of a DBMS can be detrimental compared to direct file writing.

Q7. (Scenario-Based) A bank's IT department is separating roles. Employee A manages the physical disk storage and backup strategies. Employee B designs the logical tables and ER diagrams. Employee C writes web applications that query the database. Who represents the Database Administrator (DBA)?

A) Employee A
B) Employee B
C) Employee C
D) Both B and C

> A Database Administrator (DBA) is primarily responsible for the physical realization of the database, performance tuning, backups, security, and hardware allocation. Employee B is a Database Designer, and C is an Application Developer.

Q8. (Conceptual) Which of the following is NOT a fundamental property of a Database Management System?

A) It provides mechanisms for backup and recovery.
B) It allows multiple users to access data concurrently.
C) It represents complex relationships among data.
D) It compiles backend application logic into machine code.

> A DBMS manages data storage, retrieval, and integrity. It does not compile application software (like Java or Python code) into machine code; that is the job of a programming language compiler.

Q9. (Comparison) What is the primary difference between a "Data Definition Language (DDL)" and a "Data Manipulation Language (DML)"?

A) DDL manages user permissions; DML manages backups.
B) DDL defines the database structure and schemas; DML is used to retrieve, insert, update, or delete actual data.
C) DDL is used by end-users; DML is used exclusively by DBAs.
D) DDL operates on RAM; DML operates on hard disks.

> DDL (CREATE, ALTER, DROP) alters the metadata (the schema structure). DML (SELECT, INSERT, UPDATE, DELETE) alters the instances (the actual row data inside the structures).

Q10. (Scenario-Based) An e-commerce system uses a DBMS to ensure that if a server crashes during a checkout process (after deducting inventory but before charging the credit card), the inventory deduction is undone. This capability falls under:

A) Integrity Constraints

B) Security and Authorization

C) Transaction Management and Recovery

D) Query Optimization

> Transaction Management ensures the "Atomicity" property of ACID. A transaction must execute completely (All) or not at all (Nothing). If a crash occurs mid-transaction, the recovery subsystem uses logs to undo the partial changes.

Deep Dive: Technical Internals (How a DBMS Engine Actually Works)

To truly understand why a DBMS is superior to a standard OS File System, we have to look at how it manipulates memory and disk at the hardware level:

- **Slotted Page Architecture:** Unlike file systems that read continuous streams of bytes, a DBMS divides the hard drive into fixed-size blocks called *Pages* (typically 8KB in PostgreSQL or 16KB in MySQL InnoDB). Each page uses a "Slotted Page Layout." The page header contains metadata, followed by an array of line pointers growing downward, while the actual row data (tuples) grows upward from the bottom. This allows the DBMS to delete rows and dynamically defragment the page without shifting all other rows.
- **Bypassing the OS Cache (O_DIRECT):** Modern Operating Systems try to cache file reads in RAM. However, a high-performance DBMS explicitly tells the OS to bypass this cache (using flags like `O_DIRECT` in Linux). Why? Because the DBMS's own **Buffer Manager** understands database workloads better than the OS. The DBMS knows that an "Index Root Node" page should be kept in RAM forever, while a "Sequential Scan Table" page should be evicted immediately after reading.
- **Write-Ahead Logging (WAL) & Sequential I/O:** Hard drives (especially HDDs) are incredibly slow at random writes, but fast at sequential writes. To guarantee ACID Durability without destroying performance, the DBMS does not write your `UPDATE` directly to the table's data file immediately. Instead, it writes the change to the WAL

(a sequential log file) and simply updates the page in RAM (marking it as "dirty"). Even if the power goes out a millisecond later, the database can reconstruct the exact state by replaying the sequential WAL upon reboot.

Stable Web Resources

- GeeksforGeeks: Intro to DBMS
- Wikipedia: Database

Quick Revision Cheat Sheet

- **Database:** Collection of related data.
- **DBMS:** Software to manage the database (e.g., MySQL, Oracle).
- **Metadata:** Data dictionary/catalog defining the schema.
- **Data Independence:** Application logic is immune to changes in data storage structures.
- **Redundancy:** Storing the same data twice (causes anomalies).

END OF SUBTOPIC 01 — MOVING TO SUBTOPIC 02

Subtopic 02: Database System Architecture (10 MCQs)

Quick Notes

- **ANSI/SPARC 3-Schema Architecture:**
  - *Internal Level:* Describes physical storage structures and access paths (B-Trees, Hash indexes).
  - *Conceptual Level:* Describes the structure of the whole database for a community of users (Entities, Data Types, Relationships).

- *External Level:* Describes the part of the database a specific user group is interested in (Views).

- **Data Independence:** Capacity to change the schema at one level without having to change the schema at the next higher level.
- **Client-Server Architectures:**
  - *2-Tier:* Client UI directly connects to the Database Server.
  - *3-Tier:* Client UI connects to an Application Server (Business Logic), which connects to the Database Server.
- **DBMS Components:** Query Optimizer (finds best execution plan), Buffer Manager (handles RAM caching), Transaction Manager (enforces ACID).

Common Traps & Confusions

- **Logical vs. Physical Independence Difficulty:** Logical data independence is much harder to achieve than physical data independence. Changing physical disks doesn't break apps, but altering table logic often forces application code rewrites unless views are perfectly designed.
- **External Level vs. Application UI:** Students often mistake the "External Level" for the frontend website (HTML/CSS). The External Level is actually the specific database `VIEW` or schema restricted to that frontend application, not the frontend UI itself.

MCQs

Q1. (Scenario-Based) A DBA notices that queries searching for a specific user's email are too slow. They create a B-Tree index on the `email` column. The backend application's SQL queries remain exactly the same, but execution speed increases 10x. This is a practical example of:

A) Logical Data Independence

B) Physical Data Independence

C) External Schema Mapping

D) Conceptual Transparency

> Adding an index changes the Internal Schema (the physical access path on the disk). Because the Conceptual Schema (the table) and the External views did not change, and the app didn't need to be rewritten, this demonstrates Physical Data Independence.

Q2. (Comparison) In the context of database application architecture, what is the primary distinction between a 2-Tier and a 3-Tier architecture?

A) 2-Tier systems do not use a network connection, while 3-Tier systems do.

B) 2-Tier uses NoSQL databases; 3-Tier uses Relational databases.

C) In 2-Tier, business logic is embedded in the client application; in 3-Tier, an intermediate application server handles business logic.

D) 2-Tier cannot support concurrent users, while 3-Tier can.

> In a 2-Tier model (like a thick desktop client), the UI connects via ODBC/JDBC directly to the DB, meaning the client holds the business rules. A 3-Tier model inserts a middle layer (like a Node.js or Spring Boot server) to handle business logic, drastically improving security and scalability.

Q3. (Code-Tracing) A developer runs the following command in their terminal. Which specific internal component of the DBMS architecture is responsible for generating the output shown?

postgres - query_plan

```
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 105;

-- Output:
Index Scan using idx_orders_cust on orders (cost=0.29..8.31 rows=1)
```

A) The Transaction Manager

B) The Buffer Manager

C) The Query Optimizer

D) The DDL Compiler

> The Query Optimizer analyzes the SQL statement and determines the most cost-effective execution strategy (e.g., using an Index Scan rather than a Sequential Table Scan). The `EXPLAIN` command directly reveals the Optimizer's chosen plan.

Q4. (Conceptual) In the Three-Schema Architecture, what mechanism allows the DBMS to translate a request on an External View into a request against the Conceptual Schema, and finally into a read operation on the physical disk?

A) The Data Dictionary

B) The Transaction Log

C) Mappings

D) Normalization

> Mappings are the rules/programs used by the DBMS to transform requests and results between the levels (External/Conceptual mapping and Conceptual/Internal mapping). This process of transforming requests is what actually provides Data Independence.

Q5. (Scenario-Based) An application suddenly starts throwing `SQLSyntaxErrorException` errors. Upon investigation, you discover a DBA split a massive `Customers` table into two normalized tables (`Customers_Core` and `Customers_Address`). What architectural concept failed to protect the application?

A) Physical Data Independence

B) Logical Data Independence

C) Concurrency Control

D) Network Data Independence

> Because the Conceptual schema (the tables themselves) was altered, the external application broke. If the DBA had created a `VIEW` named `Customers` that joined the two new tables, they would have successfully achieved Logical Data Independence, shielding the app from the change.

Q6. (Diagram/Visual Interpretation) If you were to draw an architectural diagram of a modern Web Application, where does the **DBMS Engine** itself typically reside?

A) Tier 1 (Presentation Layer)

B) Tier 2 (Logic Layer)

C) Tier 3 (Data Layer)

D) Scattered across all three tiers

> In a modern 3-Tier or n-Tier architecture, Tier 1 is the browser/app (Frontend), Tier 2 is the web server/API (Backend Logic), and Tier 3 is the dedicated Database Server (Data Layer) where the DBMS engine runs.

Q7. (Code-Tracing) Look at the SQL script below. The object `hr_employee_view` belongs to which level of the database architecture?

mysql - schema_setup

```sql
CREATE VIEW hr_employee_view AS
SELECT emp_id, first_name, last_name, department
FROM company.employees
WHERE status = 'ACTIVE';

GRANT SELECT ON hr_employee_view TO 'hr_intern_role';
```

A) The Internal Level

B) The Conceptual Level

C) The External Level

D) The Physical Level

> A View is a virtual table tailored to a specific group of users (in this case, an HR intern who shouldn't see inactive employees or salary data). This customized, restricted perspective is the definition of the External Level.

Q8. (Conceptual) What is the primary role of the **Storage Manager** module in a database system?

A) To act as an interface between the low-level data stored in the OS file system and the application queries submitted to the system.

B) To check user passwords and roles.

C) To compile SQL into machine code.

D) To format the output of queries into JSON or XML.

> The Storage Manager (which contains the Buffer Manager, File Manager, and Transaction Manager) handles the actual retrieval, storage, and caching of data blocks on the physical hard drives.

Q9. (Comparison) Why is **Logical Data Independence** generally considered much harder to fully achieve than **Physical Data Independence**?

A) Because physical hard drives are easier to upgrade than RAM.

B) Because SQL does not support views.

C) Because application business logic is inherently tied to the logical structure (schema) of the data, whereas physical storage details are completely hidden.

D) Because logical data independence requires object-oriented databases.

> Changing physical storage (moving from RAID 0 to RAID 5, or adding an index) is invisible to a SQL query. But if you change the logical structure (e.g., changing a 1:1 relationship to an M:N relationship), the application's underlying code and queries usually must change to handle the new logic.

Q10. (Conceptual) When a database crash occurs, the recovery manager ensures that incomplete transactions are undone and committed transactions are preserved. Which file structure, maintained by the DBMS architecture, is strictly required to accomplish this?

A) The Data Dictionary

B) The B-Tree Index file

C) The Write-Ahead Log (WAL) or System Log

D) The Query Execution Plan

> The Write-Ahead Log (WAL) records all changes made to the database *before* those changes are written to the actual data files. In the event of a crash, the recovery manager reads the WAL to Redo or Undo transactions.

Deep Dive: The Life Cycle of a Query

When an application sends a query (e.g., `SELECT * FROM Users WHERE id = 5`), it travels through the DBMS architecture in a precise sequence:

1. **Connection Manager:** Authenticates the user and checks network permissions.
2. **Parser/Compiler:** Checks the query against the Data Dictionary (Conceptual Schema) to ensure the `Users` table and `id` column actually exist, and checks syntax.
3. **Query Optimizer:** Looks at statistics. Does an index exist on `id`? If yes, it creates an execution plan to use an *Index Seek*. If no, it plans a *Full Table Scan*.
4. **Execution Engine:** Takes the Optimizer's plan and requests the specific data from the Storage Manager.
5. **Buffer Manager (Storage Level):** Checks if the page containing User ID 5 is already in RAM (Buffer Pool). If not, it issues an OS-level read to pull the block from

the physical disk (Internal Schema) into memory, then hands the data back up the chain.

Stable Web Resources

- JavaTPoint: DBMS Architecture
- Wikipedia: Data Independence

Quick Revision Cheat Sheet

- **3-Tier Architecture:** Client UI → App/Web Server → Database Engine.
- **Internal Schema:** Physical disk, indexes, block sizes.
- **Conceptual Schema:** Tables, logical constraints.
- **External Schema:** Views for end-users.
- **Physical Data Independence:** Change internal without changing conceptual.
- **Logical Data Independence:** Change conceptual without changing external.

END OF SUBTOPIC 02 — MOVING TO SUBTOPIC 03

Subtopic 03: Data Models (10 MCQs)

Quick Notes

- **Hierarchical Model:** Data organized as a strict tree. Parent-child relationship (1:N). A child can have only one parent.
- **Network Model:** Data organized as a graph using records and pointers (sets). Solves the many-to-many (M:N) limitations of the hierarchical model.
- **Relational Model:** Data organized as mathematical tables (relations). Columns = Attributes, Rows = Tuples. Based on set theory.

- **Object-Oriented Model:** Data stored as objects with state (attributes) and behavior (methods). Supports inheritance and complex types.
- **Entity-Relationship (ER) Model:** A high-level conceptual model used for database design (rectangles, diamonds, ovals) before converting to relational tables.
- **Semi-Structured / Document Model:** Data structured in flexible formats like JSON or XML, heavily used in modern NoSQL databases.

Common Traps & Confusions

- **Relational vs. ER Model:** The ER Model is strictly for *conceptual design* (drawing on a whiteboard). The Relational Model is the *logical implementation* (creating actual tables with primary keys).
- **Network Model vs. Graph Databases:** The classic Network Model (CODASYL) relied on rigid, hardcoded pointers. Modern Graph Databases (like Neo4j) use flexible nodes and edges designed for deep traversal, though both are technically graphs.

MCQs

Q1. (Conceptual) Which early database model organizes data in a strict upside-down tree structure, where every child record is allowed to have exactly one parent record?

A) Network Model
B) Relational Model
C) Hierarchical Model
D) Object-Relational Model

> The Hierarchical model connects data in a tree. For example, a "Department" node has child "Employee" nodes. Its major flaw is that an employee cannot easily belong to two departments (a Many-to-Many relationship is not natively supported).

Q2. (Comparison) How did the **Network Data Model** fundamentally improve upon the limitations of the Hierarchical Data Model?

A) It introduced the use of SQL for declarative querying.
B) It allowed a child record to have multiple parent records, forming a graph structure.

C) It stored data strictly in tabular rows and columns.

D) It introduced the concept of Object Inheritance.

> The Network model (standardized by CODASYL) solved the M:N limitation of the hierarchical model by allowing network-like graph relationships, where a single child record could link to multiple parent records using pointers.

Q3. (Scenario-Based) A software architect is designing the backend for a social media application. The most critical, high-frequency query is finding "friends of friends of friends" (3rd-degree connections). Which data model is most highly optimized for this specific traversal operation?

A) Relational Model

B) Hierarchical Model

C) Graph Data Model

D) Column-Family Data Model

> Modern Graph databases (a type of NoSQL) excel at relationship-heavy queries. In a relational database, finding 3rd-degree connections requires multiple expensive recursive JOIN operations. In a Graph DB, it is a fast, localized pointer traversal along edges.

Q4. (Conceptual) In the **Relational Data Model** introduced by E.F. Codd in 1970, what fundamental mathematical concept serves as the absolute foundation for tables?

A) Calculus matrices

B) Set theory and relations

C) Directed acyclic graphs

D) Doubly Linked Lists

> A "Relation" is a mathematical term for a set of tuples (rows). The relational model treats data strictly as sets, which allows the use of Relational Algebra (Union, Intersection, Select, Project) to declaratively query data.

Q5. (Scenario-Based) During the initial requirements phase of a new university database, a database architect draws a diagram on a whiteboard. They draw rectangles for `Students`, diamonds representing `Enrollments`, and ovals representing `Student_IDs`. What specific data model is being utilized here?

A) Physical Data Model

B) Relational Data Model

C) Entity-Relationship (ER) Model

D) Object-Oriented Model

> The ER Model is a high-level conceptual data model. It uses standard visual notations (Rectangles = Entities, Diamonds = Relationships, Ovals = Attributes) to represent business rules before writing any SQL table definitions.

Q6. (Code/Data Tracing) Examine the raw data representation extracted from a database below. What type of data model is being utilized to store this record?

db_export.json

```
{
  "_id": ObjectId("60f1b2b3"),
  "name": "Sarah Connor",
  "skills": ["Python", "Machine Learning"],
  "contact": {
    "email": "sarah@tech.com",
    "phone": null
  }
}
```

A) Relational Data Model

B) Hierarchical Database Model

C) Semi-Structured / Document Data Model

D) Network Data Model

> This snippet is JSON (JavaScript Object Notation), which represents a Semi-Structured or Document Data model (used heavily in NoSQL databases like MongoDB). Unlike the rigid relational model, it is schemaless and natively allows nested data structures (like arrays and inner objects).

Q7. (Comparison) Which of the following is a primary architectural advantage of the **Object-Oriented Data Model** over the traditional Relational Data Model?

A) It uses standard declarative SQL for all queries.

B) It has a simpler mathematical foundation based on Relational Calculus.

C) It natively supports complex data types, polymorphism, and inheritance without requiring translation.

D) It is completely immune to data redundancy and concurrency issues.

> The Object-Oriented Database Management System (OODBMS) allows developers to store objects exactly as they exist in OOP languages (Java/C++/Python), including inheritance. This avoids the "Object-Relational Impedance Mismatch" where objects must be flattened into tables.

Q8. (Conceptual) Database design typically progresses through three distinct modeling phases. Which data model specifies the absolute low-level storage details, such as record formats, file structures, table spaces, and B-Tree access paths?

A) Conceptual Data Model

B) Logical Data Model

C) Physical Data Model

D) External Data Model

> Database design follows three phases: Conceptual (ER diagram representing business logic), Logical (Relational tables and foreign keys), and Physical (Database-specific syntax, index creation, disk partitioning, and file organization).

Q9. (Scenario-Based) A legacy banking system runs on an old Hierarchical database. A customer can have multiple accounts (checking, savings). However, the bank is trying to implement "Joint Accounts" where two distinct customers share one account. The development team reports this is extremely difficult. Why?

A) The hierarchical model strictly prohibits a child node (Account) from having more than one parent node (Customer).

B) The hierarchical model does not support integer data types required for financial balances.

C) The database lacks primary keys, which are required for any relationship.

D) Hierarchical databases cannot store more than a certain amount of records per tree branch.

> Because the hierarchical model uses a strict tree topology, 1:N relationships are easy. However, M:N relationships (Many customers mapped to Many accounts, or multiple parents for one child) break the tree structure, requiring messy, redundant data duplication to simulate.

Q10. (Conceptual) In the context of database models and architecture, what specific problem does "Data Abstraction" solve for the end-user?

A) It automatically encrypts data so unauthorized users cannot read it over the network.

B) It hides complex low-level storage and hardware details, presenting a simplified logical view of the data.

C) It automatically deletes duplicate records from a table during an INSERT.

D) It converts relational tables into NoSQL document stores on the fly.

> Data Abstraction allows a developer to write a simple `SELECT * FROM Users` without needing to know exactly how the operating system reads the magnetic disk sectors, or how it physically traverses a B-Tree index structure to locate that data.

Deep Dive: The Object-Relational Impedance Mismatch

Modern software applications are almost universally written in Object-Oriented (OO) languages (Java, C#, Python). However, their data is typically persisted in Relational databases (SQL). This creates a massive architectural friction known as the **Object-Relational Impedance Mismatch**.

- **Structure Difference:** OO uses nested objects, lists, and graphs. Relational uses flat tables.
- **Identity:** OO identity is based on memory reference (e.g., `obj1 == obj2`). Relational identity is based on Primary Keys.
- **Inheritance:** OO naturally supports inheritance (e.g., `Manager extends Employee`). Relational DBs have no native concept of inheritance, requiring workarounds like Single Table Mapping or Joined Table Mapping.

To bridge this gap, developers use **Object-Relational Mapping (ORM)** frameworks (like Hibernate for Java, Entity Framework for C#, or SQLAlchemy for Python) that automatically translate OO logic into complex SQL queries behind the scenes.

Stable Web Resources

- GeeksforGeeks: Data Models in DBMS
- Wikipedia: Database Model

Quick Revision Cheat Sheet

- **Hierarchical Model:** Tree structure, 1 parent per child. Rigid.
- **Network Model:** Graph structure, multiple parents allowed (CODASYL).
- **Relational Model:** Flat Tables, Rows (tuples), Columns (attributes). Based on Set Theory.
- **ER Model:** High-level conceptual whiteboard design (Entities & Relationships).
- **Document/NoSQL Model:** JSON/XML, flexible schemaless structures.
- **Graph Model:** Nodes and Edges, optimized for relationship traversal (e.g., Neo4j).

END OF SUBTOPIC 03 — MOVING TO SUBTOPIC 04

Subtopic 04: Relational Database Concepts (15 MCQs)

Quick Notes

- **Relation:** A mathematical set of tuples (rows). In SQL, this is represented as a Table.
- **Attribute:** A named column in a relation. The **Domain** is the set of allowable values for an attribute.
- **Tuple:** A single row in a relation representing a specific entity instance.
- **Degree:** The total number of attributes (columns) in a relation.
- **Cardinality:** The total number of tuples (rows) currently in a relation.
- **Keys:**
  - *Super Key:* A set of one or more attributes that uniquely identifies a tuple.
  - *Candidate Key:* A minimal Super Key (no redundant attributes).
  - *Primary Key:* The specific Candidate Key chosen by the DBA to uniquely identify rows.
  - *Foreign Key:* An attribute that matches the Primary Key of another table to create a relationship.
- **Integrity Constraints:** Entity Integrity (PK cannot be NULL) and Referential Integrity (FK must match a PK or be NULL).

Common Traps & Confusions

- **Candidate Key vs. Super Key:** All Candidate Keys are Super Keys, but not all Super Keys are Candidate Keys. If `{ID}` is a key, then `{ID, Name}` is a Super Key, but NOT a Candidate Key because `Name` is redundant for uniqueness.
- **Primary Key vs. Unique Key:** A table can have only one Primary Key, and it strictly cannot contain NULL values. A table can have multiple Unique Keys, and in many systems (like PostgreSQL/MySQL), they *can* contain NULLs.

- **Foreign Key NULLs:** Foreign Keys *can* be NULL unless explicitly marked `NOT NULL`. A NULL Foreign Key represents an optional relationship.

MCQs

Q1. (Calculation) Consider a relation `Employees(EmpID, Name, DeptID, Salary, Phone)`. Currently, there are 500 employee records. What are the **Degree** and the **Cardinality** of this relation?

A) Degree: 500, Cardinality: 5

B) Degree: 5, Cardinality: 500

C) Degree: 4, Cardinality: 500

D) Degree: 5, Cardinality: 1

> Degree refers to the number of columns (attributes). Here, there are 5 columns. Cardinality refers to the number of rows (tuples). Here, there are 500 rows.

Q2. (Conceptual) According to the **Entity Integrity Constraint**, which of the following statements is true regarding a Primary Key?

A) A Primary Key can be NULL if the row is not referenced by any Foreign Key.

B) A Primary Key must be an integer.

C) No component of a Primary Key is allowed to accept a NULL value.

D) A Primary Key must be a single attribute only.

> Entity Integrity states that no part of the primary key can be null. This ensures that every tuple can be uniquely identified; a null value would imply an unknown identity, which violates the fundamental requirement of a key.

Q3. (Scenario-Based) A database designer identifies three possible unique identifiers for a `Vehicle` table: `VIN` (Vehicle Identification Number), `EngineNumber`, and `LicensePlate`. In relational terminology, what are these three attributes called before one is officially selected as the Primary Key?

A) Super Keys

B) Secondary Keys

C) Candidate Keys

D) Foreign Keys

> Candidate Keys are the set of all minimal super keys that are eligible to become the Primary Key. Once the designer chooses one (e.g., VIN), the others are referred to as Alternate Keys.

Q4. (Code-Tracing) A developer attempts to execute the following SQL script. Which relational constraint will likely cause the second `INSERT` to fail?

sql-shell - constraints_test

```sql
CREATE TABLE Dept (
    dept_id INT PRIMARY KEY,
    dname VARCHAR(50)
);

CREATE TABLE Emp (
    emp_id INT PRIMARY KEY,
    ename VARCHAR(50),
    d_ref INT,
    FOREIGN KEY (d_ref) REFERENCES Dept(dept_id)
);

INSERT INTO Dept VALUES (10, 'IT');
INSERT INTO Emp VALUES (101, 'Alice', 20);
```

A) Entity Integrity Constraint

B) Referential Integrity Constraint

C) Domain Constraint

D) Key Constraint

> The table `Emp` has a foreign key `d_ref` that references `Dept(dept_id)`. Since the value `20` does not exist in the `Dept` table (only 10 exists), the insertion violates Referential Integrity.

Q5. (Comparison) What is the technical difference between a **Super Key** and a **Candidate Key**?

A) Super Keys are used for relationships, Candidate Keys are used for indexing.

B) Super Keys must be numeric, Candidate Keys can be strings.

C) A Candidate Key is a Super Key with no redundant attributes (minimal); a Super Key may contain unnecessary attributes.

D) Super Keys are chosen by the DBMS; Candidate Keys are chosen by the user.

> A Super Key is any set of attributes that uniquely identifies a row. A Candidate Key is the "minimal" version. For example, if {PassportNumber} is unique, then {PassportNumber, EyeColor} is a Super Key, but not a Candidate Key because EyeColor is redundant.

Q6. (Scenario-Based) A table `ProjectAssignments` uses a composite Primary Key consisting of `(EmployeeID, ProjectID)`. If a manager tries to assign Employee 501 to Project 'A' twice, which error will the DBMS throw?

A) Referential Integrity Violation

B) Primary Key Violation (Duplicate Key)

C) Check Constraint Violation

D) Domain Integrity Violation

> In a composite key, the combination of the two values must be unique. Inserting `(501, 'A')` twice violates the uniqueness requirement of the Primary Key.

Q7. (Conceptual) In the relational model, why is it stated that "tuples are unordered"?

A) Because databases sort data randomly for security.

B) Because a relation is mathematically defined as a set, and sets do not have an inherent sequence.

C) Because hardware cannot store data in a specific order.

D) Because ordering data makes the `SELECT` query too slow.

> Mathematically, a relation is a set of tuples. By definition, a set is an unordered collection of distinct elements. While SQL might return rows in the order they were inserted, the relational model dictates that the order must not be relied upon unless an `ORDER BY` clause is used.

Q8. (Code-Tracing) Consider the following table and data. What happens if you try to execute: `DELETE FROM Parent WHERE id = 1;`?

postgres - delete_logic

```
CREATE TABLE Parent (id INT PRIMARY KEY);
CREATE TABLE Child (
    c_id INT PRIMARY KEY,
    p_id INT REFERENCES Parent(id) ON DELETE CASCADE
);

INSERT INTO Parent VALUES (1);
```

```sql
INSERT INTO Child VALUES (100, 1);
```

A) The error "Foreign key constraint violation" is thrown.

B) The row in Parent is deleted, but the row in Child remains with a NULL p_id.

C) The row in Parent is deleted, and the row in Child is automatically deleted as well.

D) The command is ignored because a child exists.

> The `ON DELETE CASCADE` option tells the DBMS that if a referenced row in the Parent table is deleted, all rows in the Child table that reference it should also be deleted automatically.

Q9. (Comparison) How does a **Foreign Key** differ from a **Primary Key** regarding NULL values?

A) Both must be NOT NULL.

B) Primary Keys can be NULL; Foreign Keys cannot.

C) Primary Keys must be NOT NULL; Foreign Keys can be NULL to represent an optional relationship.

D) Neither can be NULL in a relational database.

> Entity Integrity forbids NULLs in Primary Keys. However, a Foreign Key can be NULL if the relationship is optional (e.g., an Employee who is not currently assigned to any Department).

Q10. (Calculation) A relation `R` has attributes `{A, B, C, D}`. The only unique combinations of attributes are `{A, B}` and `{A, C}`. How many Candidate Keys does this relation have?

A) 1

B) 2

C) 3

D) 4

> Since both `{A, B}` and `{A, C}` are minimal sets that uniquely identify tuples, they are both Candidate Keys. Attributes `B` and `C` are part of different keys.

Q11. (Conceptual) What does the term **"Domain"** represent in a relational schema?

A) The physical location of the database on the server.

B) The set of all possible atomic values that can be assigned to an attribute.

C) The group of users who have access to a table.

D) The number of tables in the database.

> A Domain is a pool of values from which an attribute takes its values. For example, the domain for a Age attribute might be `integers between 0 and 120`.

Q12. (Scenario-Based) An e-commerce database has a table `Orders`. The designer wants to ensure that the `total_amount` column never accepts negative values. Which relational concept should be applied?

A) Referential Integrity

B) Entity Integrity

C) Domain Constraint (using CHECK)

D) Foreign Key Constraint

> A Domain Constraint restricts the values allowed in a column. In SQL, this is typically implemented using a `CHECK` constraint (e.g., `CHECK (total_amount >= 0)`).

Q13. (Calculation) If Relation A has 10 tuples and Relation B has 5 tuples, what is the **Cardinality** of the Cartesian Product (A x B)?

A) 15

B) 10

C) 50

D) 5

> The Cartesian Product of two sets contains every possible combination of elements. In database terms, the resulting cardinality is the product of the cardinalities of the two relations: 10 * 5 = 50.

Q14. (Code-Tracing) Observe the SQL below. What is the Primary Key of the `Enrollments` table?

sql-editor - composite_keys

```sql
CREATE TABLE Enrollments (
    student_id INT,
```

```
    course_id INT,
    semester VARCHAR(10),
    PRIMARY KEY (student_id, course_id)
);
```

A) student_id

B) course_id

C) A composite key consisting of (student_id, course_id)

D) student_id, course_id, and semester

> A Primary Key can consist of multiple columns. This is known as a composite key. In this case, neither student_id nor course_id alone is the PK; only their combination is.

Q15. (Comparison) What happens if you try to insert a duplicate value into a column that has a **UNIQUE** constraint, but is NOT the Primary Key?

A) The insert succeeds because it is not the Primary Key.

B) The insert fails because the UNIQUE constraint enforces uniqueness just like a Primary Key.

C) The database automatically increments the value to make it unique.

D) The old record is overwritten by the new one.

> Both PRIMARY KEY and UNIQUE constraints enforce uniqueness. The primary difference is that a table can have only one PK and it cannot be NULL, whereas it can have many UNIQUE columns.

Deep Dive: Relational Integrity & The NULL Problem

Relational theory is based on **Predicate Logic**. A table represents a predicate (e.g., "Person X is an employee in Department Y"). When we insert a row, we are asserting that the predicate is TRUE for those specific values. However, real-world data is often missing or unknown, leading to the introduction of **NULL**.

- **The 3-Valued Logic (3VL):** In standard logic, something is either True or False. Because of NULLs, SQL uses 3VL: True, False, and **Unknown**. For example, `NULL = NULL` does not return True; it returns Unknown. This is why you must use `IS NULL` instead of `= NULL`.

- **Referential Integrity Mechanics:** When a Foreign Key is defined, the DBMS doesn't just check it during `INSERT`. It must also monitor the Parent table. There are four standard behaviors for `ON DELETE` and `ON UPDATE`:
  1. **RESTRICT/NO ACTION:** Prevents deletion of the parent if children exist (The default).
  2. **CASCADE:** Deletes the children when the parent is deleted.
  3. **SET NULL:** Sets the child's FK column to NULL when the parent is deleted.
  4. **SET DEFAULT:** Sets the child's FK column to its default value.
- **Codd's Rule #10 (Physical Data Independence):** This rule states that the logical level of the database (how you write your SQL) must not be impacted by how the data is physically stored. This is why we use Primary Keys rather than "Memory Addresses" or "Row IDs" to link tables.

Stable Web Resources

- [Wikipedia: Relational Model]

Quick Revision Cheat Sheet

- **Degree:** Number of Columns. **Cardinality:** Number of Rows.
- **Super Key:** Any unique identifier.
- **Candidate Key:** Minimal Super Key.
- **Primary Key:** Chosen Candidate Key (No NULLs allowed).
- **Foreign Key:** Links to a PK in another table (Allows NULLs).
- **Entity Integrity:** PK cannot be NULL.
- **Referential Integrity:** FK must match an existing PK or be NULL.

END OF SUBTOPIC 04 — MOVING TO SUBTOPIC 05

Subtopic 05: Relational Algebra & Calculus (12 MCQs)

Quick Notes

- **Relational Algebra:** A procedural query language. It tells the system *how* to retrieve the data using a sequence of operations.
- **Fundamental Operations:**
  - *Selection (σ):* Filters tuples (rows) based on a condition.
  - *Projection (π):* Selects specific attributes (columns) and removes duplicates.
  - *Union (∪):* Combines tuples from two relations (must be Union Compatible).
  - *Set Difference (-):* Tuples in relation A but not in B.
  - *Cartesian Product (×):* Combines every tuple of A with every tuple of B.
  - *Rename (ρ):* Changes the name of a relation or its attributes.
- **Relational Calculus:** A non-procedural (declarative) language. It describes *what* data is needed without specifying the steps to get it.
- **Tuple Relational Calculus (TRC):** Filters based on tuples (e.g., {t | P(t)}).
- **Domain Relational Calculus (DRC):** Filters based on attribute domains (e.g., {<x1, x2> | P(x1, x2)}).

Common Traps & Confusions

- **Union Compatibility:** Two relations are only union-compatible if they have the same number of attributes (degree) and the domains of corresponding attributes are identical. You cannot union a `Users` table with an `Orders` table directly.
- **Duplicates:** In pure Relational Algebra, operations like Projection automatically remove duplicate rows. In SQL, you must explicitly use `DISTINCT` to achieve this.
- **Procedural vs. Declarative:** Remember that Algebra is like a recipe (steps), while Calculus is like a wishlist (the result). SQL is primarily based on Relational Calculus but executed using Relational Algebra.

MCQs

Q1. (Conceptual) Which of the following best describes the fundamental difference between Relational Algebra and Relational Calculus?

A) Algebra is used for NoSQL, while Calculus is used for SQL.

B) Algebra is declarative, while Calculus is procedural.

C) Algebra is procedural (specifies how to get data), while Calculus is declarative (specifies what data to get).

D) Algebra can only handle one table at a time, while Calculus can handle many.

> Relational Algebra provides a set of operations (Select, Project, Join) that are executed in a specific order to produce a result. Relational Calculus simply describes the properties that the resulting data must satisfy.

Q2. (Calculation) Relation R has 10 tuples and Relation S has 20 tuples. If we perform the operation **R** ∪ **S**, what is the **minimum** and **maximum** possible number of tuples in the result? (Assume R and S are union-compatible).

A) Min: 10, Max: 20

B) Min: 10, Max: 30

C) Min: 20, Max: 30

D) Min: 0, Max: 30

> Maximum: If R and S have no common tuples, the result is 10 + 20 = 30. Minimum: If all 10 tuples of R are already present in S, the result is simply the size of S, which is 20 (since sets remove duplicates).

Q3. (Notation-Tracing) A developer writes the following algebraic expression to retrieve data from a `Products` table:

$\pi_{\text{name, price}} (\sigma_{\text{category = 'Electronics'}} (\text{Products}))$

In plain English, what does this expression do?

A) Updates the price of all Electronics.

B) Selects all columns for products where the name is 'Electronics'.

C) Displays only the names and prices of products belonging to the 'Electronics' category.

D) Deletes all products that are not Electronics.

> The inner operation σ (Sigma) filters the rows where category is 'Electronics'. The outer operation π (Pi) limits the output columns to 'name' and 'price'.

Q4. (Conceptual) For the **Set Difference (R - S)** operation to be valid, which of the following conditions must be met?

A) R and S must have a primary-foreign key relationship.

B) R must have more tuples than S.

C) R and S must be Union-Compatible (same degree and compatible domains).

D) S must be a subset of R.

> Set operations (Union, Intersection, Difference) require the participating relations to have the same structure (number of columns and corresponding data types). This is called Union Compatibility.

Q5. (Scenario-Based) A marketing team wants a list of all `Customers` who have **never** placed an `Order`. Which relational algebra operator is best suited for this task?

A) Cartesian Product (×)

B) Union (∪)

C) Set Difference (-)

D) Natural Join (⋈)

> To find customers with no orders, you take the set of all Customer IDs and subtract the set of Customer IDs that appear in the Orders table: π_{cust_id}(Customers) - π_{cust_id}(Orders).

Q6. (Code-Tracing) Trace the logic of the following **Tuple Relational Calculus** expression: **{ t | t ∈ Employees ∧ t.salary > 5000 }**

calculus-engine

```
// Relation: Employees
// id | name  | salary
// 1  | John  | 4000
// 2  | Sarah | 6000
// 3  | Mike  | 5500
```

A) Returns all columns for John.

B) Returns all columns for Sarah and Mike.

C) Returns only the salary column for everyone.

D) Returns a count of employees.

> The expression defines a set of tuples 't' such that 't' is a member of the Employees relation and the salary attribute of 't' is greater than 5000. Sarah and Mike both meet this criteria.

Q7. (Calculation) Relation R has degree 3 (A, B, C) and 5 tuples. Relation S has degree 2 (D, E) and 4 tuples. What is the **Degree** and **Cardinality** of the Cartesian Product **R × S**?

A) Degree: 6, Cardinality: 9

B) Degree: 5, Cardinality: 9

C) Degree: 5, Cardinality: 20

D) Degree: 6, Cardinality: 20

> Degree of R × S = Degree(R) + Degree(S) = 3 + 2 = 5.
> Cardinality of R × S = Cardinality(R) * Cardinality(S) = 5 * 4 = 20.

Q8. (Conceptual) Why is the **Rename (ρ)** operator considered essential in Relational Algebra, particularly during a Self-Join scenario?

A) To change the physical name of the table on the hard drive.

B) To distinguish between different instances of the same relation when joining a table with itself.

C) To bypass primary key constraints.

D) To convert a table into a view.

> When joining a table to itself (e.g., finding employees who earn more than their managers within the same Employee table), we must use the Rename operator to give the table two different aliases so the engine can distinguish the attributes.

Q9. (Scenario-Based) A query in Relational Calculus is said to be **"Unsafe"** if it can potentially return an infinite number of tuples. Which of the following is an example of an unsafe expression?

A) { t | t ∈ Users ∧ t.id = 5 }

B) { t | ¬(t ∈ Users) }

C) { t | t ∈ Users ∨ t ∈ Admins }

D) { t | t.name = 'Alice' ∧ t ∈ Users }

> An unsafe expression like "all tuples not in the Users table" is problematic because the universe of possible tuples is infinite. Safe expressions are those that only return values present in the database domains.

Q10. (Notation-Tracing) In Relational Algebra, which operator is mathematically equivalent to a **Cartesian Product followed by a Selection** based on a join condition?

A) Projection (π)

B) Union (∪)

C) Theta Join ($\bowtie_\theta$)

D) Set Difference (-)

> A Theta Join is specifically defined as $\sigma_{condition}(R \times S)$. It combines the tables and then immediately filters them based on the provided predicate (θ).

Q11. (Comparison) What is the primary difference between **Tuple Relational Calculus (TRC)** and **Domain Relational Calculus (DRC)**?

A) TRC uses SQL, DRC uses XML.

B) TRC uses variables that range over entire rows (tuples); DRC uses variables that range over individual column values (domains).

C) TRC is faster than DRC.

D) DRC is only used for mathematical proofs, not real databases.

> In TRC, you query for rows: {t | Employee(t)}. In DRC, you query for components: {<id, name, sal> | Employee(id, name, sal) ∧ sal > 5000}. Both are equally powerful (Relational Completeness).

Q12. (Calculation) Relation R(A, B) has 3 tuples: {(1, 2), (3, 4), (5, 6)}. Relation S(B, C) has 2 tuples: {(2, 7), (4, 8)}. What is the result of the **Natural Join (R ⋈ S)**?

A) {(1, 2, 7), (3, 4, 8), (5, 6, NULL)}

B) {(1, 2, 7), (3, 4, 8)}

C) {(1, 2, 2, 7), (3, 4, 4, 8)}

D) Empty Set

> A Natural Join looks for common attributes (here, attribute 'B'). It joins tuples where the value of B matches. (1,2) joins with (2,7) to become (1,2,7). (3,4) joins with (4,8) to become (3,4,8). (5,6) has no match in S, so it is excluded.

Deep Dive: Relational Completeness

A query language is said to be **Relationally Complete** if it can express any query that can be formulated in the fundamental Relational Algebra. Both Tuple Relational Calculus and Domain Relational Calculus are relationally complete.

The Query Optimizer Secret: Even though we write SQL (which is declarative/ Calculus), the DBMS does not execute it directly as a "wishlist." Instead, the **Query Optimizer** parses the SQL and translates it into an internal **Relational Algebra Tree**. This tree represents the procedural steps. The optimizer then rearranges the nodes of this tree (e.g., performing Selection as early as possible to reduce the number of rows) to find the most efficient path. This process is known as **Algebraic Query Optimization**.

Stable Web Resources

- Wikipedia: Relational Algebra
- GeeksforGeeks: Relational Calculus

Quick Revision Cheat Sheet

- **σ (Sigma):** Select Rows. **π (Pi):** Project Columns.
- **Union (∪):** Combine (Requires Union Compatibility).
- **Difference (-):** In A, not in B.
- **Join (⋈):** Cartesian Product + Selection.
- **Algebra:** Procedural query language (How).
- **Calculus:** Declarative query language (What).
- **Safe Expression:** Returns a finite result from database domains.

END OF SUBTOPIC 05 — MOVING TO SUBTOPIC 06

Subtopic 06: Structured Query Language (SQL) (20 MCQs)

Quick Notes

- **SQL Categories:** DDL (Structure), DML (Data), DCL (Security), TCL (Transactions), DQL (Retrieval).
- **SELECT Statement:** The primary DQL command. Syntax order: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY.
- **NULL Handling:** NULL represents a missing value. Standard comparisons (`= NULL`) always fail; use `IS NULL` or `IS NOT NULL`.
- **Wildcards:** In the `LIKE` operator, `%` matches zero or more characters, and _ matches exactly one character.
- **Aggregate Functions:** `COUNT()`, `SUM()`, `AVG()`, `MIN()`, and `MAX()`. They generally ignore NULL values (except `COUNT(*)`).

Common Traps & Confusions

- **DELETE vs. TRUNCATE:** `DELETE` is DML (logged per row, can have a WHERE clause). `TRUNCATE` is DDL (drops data pages, faster, cannot be filtered).
- **Precedence of AND/OR:** `AND` has higher precedence than `OR`. Always use parentheses to ensure logic like `(A OR B) AND C` works as intended.
- **Counting NULLs:** `COUNT(column_name)` counts only non-null values. `COUNT(*)` counts every row regardless of content.
- **WHERE vs. HAVING:** `WHERE` filters raw rows before aggregation. `HAVING` filters grouped results after aggregation.

MCQs

Q1. (Code-Tracing) A developer executes the following query on a `Products` table. What will be the final calculated value in the `discounted_price` column for a product with a `price` of 100 and a `tax` of 10?

mysql - query_editor

```mysql
SELECT name,
       price + tax * 0.90 AS discounted_price
FROM Products;
```

A) 99.00

B) 90.00

C) 109.00

D) 110.00

> Operator precedence applies in SQL. Multiplication (*) happens before Addition (+). Calculation: `100 + (10 * 0.90)` = `100 + 9` = `109`. To apply the discount to the total, parentheses would be needed: `(price + tax) * 0.90`.

Q2. (Conceptual) Which of the following SQL keywords is strictly required to remove duplicate rows from the result set of a `SELECT` query?

A) UNIQUE

B) DISTINCT

C) GROUP BY

D) ALL

> The `DISTINCT` keyword follows `SELECT` to ensure that only unique combinations of values are returned in the output. While `GROUP BY` can achieve similar results, `DISTINCT` is the standard for duplicate removal in DQL.

Q3. (Scenario-Based) A marketing database contains a `Customers` table. You need to find all customers whose names start with 'S' and end with 'n' (e.g., 'Steven', 'Susan'). Which `WHERE` clause is correct?

A) WHERE name LIKE 'S_n'

B) WHERE name = 'S%n'

C) WHERE name LIKE 'S%n'

D) WHERE name IN ('S%', '%n')

> The `LIKE` operator uses `%` as a wildcard for any sequence of characters. `'S%n'` means 'S' followed by anything, ending with 'n'. `'S_n'` would only match three-letter names like 'Sun'.

Q4. (Calculation) A table `Survey` has a column `feedback`. There are 10 rows total: 7 rows have text comments, and 3 rows have NULL values. What will be the result of `SELECT COUNT(feedback) FROM Survey;`?

A) 10

B) 7

C) 3

D) 0

> Aggregate functions like `COUNT(column_name)` ignore NULL values and only count rows where data is present. `COUNT(*)` would have returned 10.

Q5. (Code-Tracing) Observe the following logical conditions in a `WHERE` clause. Which rows will be returned?

sql-logic-test

```sql
-- Table Data: (id, category, stock)
-- 1, 'A', 10
-- 2, 'B', 50
-- 3, 'A', 100

SELECT * FROM inventory
WHERE category = 'A' OR category = 'B' AND stock > 60;
```

A) Rows 2 and 3

B) Rows 1 and 3

C) Only Row 3

D) All 3 Rows

> `AND` has higher precedence than `OR`. The query is interpreted as: `(category = 'A') OR (category = 'B' AND stock > 60)`.
> Row 1: True OR (False) = **True**.
> Row 2: False OR (True AND False) = **False**.
> Row 3: True OR (False) = **True**.

Q6. (Scenario-Based) A database administrator needs to change the data type of the `phone_number` column from `INT` to `VARCHAR(20)` in an existing `Contacts` table. Which SQL command should be used?

A) UPDATE Contacts SET phone_number = VARCHAR(20);

B) MODIFY TABLE Contacts phone_number VARCHAR(20);

C) ALTER TABLE Contacts ALTER COLUMN phone_number VARCHAR(20);

D) CHANGE Contacts phone_number TO VARCHAR(20);

> The `ALTER TABLE` command is used to modify the structure of an existing table (DDL). The specific sub-syntax varies slightly by engine (e.g., `MODIFY` in MySQL, `ALTER COLUMN` in PostgreSQL/SQL Server), but `ALTER TABLE` is the root command.

Q7. (Conceptual) Why is the `TRUNCATE` command generally much faster than the `DELETE` command for removing all rows from a large table?

A) TRUNCATE uses multiple threads while DELETE is single-threaded.

B) TRUNCATE deallocates data pages and generates minimal logs; DELETE scans and logs every row removal.

C) TRUNCATE only works on indexed columns.

D) DELETE requires the database to be in offline mode.

> `TRUNCATE` is a DDL operation that records the deallocation of data pages in the transaction log rather than logging every individual row deletion. This drastically reduces I/O and overhead.

Q8. (Code-Tracing) What is the result of the following query if the `salary` column contains a NULL for the employee 'Bob'?

sql-null-trap

```sql
SELECT name FROM Employees
WHERE salary <> 5000 OR salary = 5000;
```

A) Returns all employees including Bob.

B) Returns all employees except Bob.

C) Returns only Bob.

D) Throws a syntax error.

> In SQL, `NULL` compared to any value (even `NULL` itself) using standard operators (=, <>, >) results in `UNKNOWN`. Since the `WHERE` clause only returns rows where the condition is `TRUE`, Bob's row is filtered out because `NULL <> 5000` is UNKNOWN.

Q9. (Calculation) You need to sort a list of students by `grade` (highest first) and then by `last_name` alphabetically for students with the same grade. Which clause is correct?

A) ORDER BY grade, last_name DESC

B) ORDER BY last_name, grade DESC

C) ORDER BY grade DESC, last_name ASC

D) SORT BY grade DESC, last_name ASC

> `DESC` applies only to the attribute immediately preceding it. To sort grade descending and last_name ascending, we specify `grade DESC` followed by `last_name` (ASC is the default).

Q10. (Scenario-Based) A banking application needs to find all accounts that do **not** have a `secondary_owner` assigned. Which SQL condition is mathematically correct?

A) WHERE secondary_owner = NULL

B) WHERE secondary_owner != NULL

C) WHERE secondary_owner IS NULL

D) WHERE secondary_owner IS EMPTY

> NULL represents the absence of a value. It cannot be checked using = because `NULL = NULL` is unknown. The special operator `IS NULL` is provided for this purpose.

Q11. (Conceptual) In SQL, which of the following is true about the `BETWEEN` operator (e.g., `WHERE price BETWEEN 10 AND 20`)?

A) It is exclusive (excludes 10 and 20).

B) It is inclusive (includes both 10 and 20).

C) It only works with integers.

D) It is faster than using >= and <=.

> The `BETWEEN` operator is shorthand for `val >= low AND val <= high`. It is always inclusive of the boundary values provided.

Q12. (Code-Tracing) Trace the output of this update operation. How many rows are affected?

postgres - update_test

```
-- Table: score(id, val)
-- (1, 10), (2, 20), (3, 30), (4, 40)
```

```sql
UPDATE score SET val = val + 5
WHERE id > 1 AND val < 40;
```

A) 1 row

B) 2 rows

C) 3 rows

D) 4 rows

> Row 1 (id=1): Fails `id > 1`.
> Row 2 (id=2, val=20): Passes both.
> Row 3 (id=3, val=30): Passes both.
> Row 4 (id=4, val=40): Fails `val < 40`.
> Total affected: 2 rows (IDs 2 and 3).

Q13. (Scenario-Based) You need to add a new employee to the database, but you only know their `name` and `department`. The `salary` should be left for later. Which SQL syntax is correct?

A) INSERT INTO Emp (name, dept) VALUES ('John', 'Sales');

B) INSERT INTO Emp VALUES ('John', 'Sales');

C) ADD TO Emp (name, dept) VALUES ('John', 'Sales');

D) INSERT INTO Emp (name, dept, salary) VALUES ('John', 'Sales');

> When only providing some columns, you must list the column names in parentheses after the table name. If the columns are omitted, SQL expects values for every column in the table in their defined order.

Q14. (Calculation) What is the result of `SELECT AVG(val) FROM T;` if table `T` has values: `10, 20, NULL, 30`?

A) 15

B) 20

C) 22.5

D) NULL

> The `AVG` function ignores NULLs. The calculation is `(10 + 20 + 30) / 3` (three non-null values). `60 / 3 = 20`.

Q15. (Conceptual) The `AS` keyword in SQL is primarily used for:

A) Defining a Primary Key.

B) Sorting data in ascending order.

C) Aliasing a column or table name to provide a temporary, more readable name.

D) Specifying the data type of a variable.

> `AS` is the alias keyword. It allows you to rename output columns for clarity (e.g., `SELECT price * tax AS total_cost`) or rename tables for brevity in complex joins.

Q16. (Code-Tracing) Which query returns exactly the same result as: `SELECT * FROM Items WHERE id IN (1, 2, 3);`?

A) SELECT * FROM Items WHERE id = 1 AND id = 2 AND id = 3;

B) SELECT * FROM Items WHERE id = 1 OR id = 2 OR id = 3;

C) SELECT * FROM Items WHERE id > 1 AND id < 3;

D) SELECT * FROM Items WHERE id BETWEEN 1 AND 2;

> The `IN` operator is a logical shorthand for multiple `OR` conditions on the same attribute. The `AND` option (A) would return nothing because an `id` cannot be three different values simultaneously.

Q17. (Scenario-Based) An HR database needs to delete all employees who belong to the 'Temporary' department. Which statement is correct?

A) TRUNCATE FROM Employees WHERE dept = 'Temporary';

B) REMOVE FROM Employees WHERE dept = 'Temporary';

C) DELETE FROM Employees WHERE dept = 'Temporary';

D) DROP Employees WHERE dept = 'Temporary';

> `DELETE` is the standard DML command for removing specific rows based on a condition. `TRUNCATE` and `DROP` are DDL and do not support `WHERE` clauses.

Q18. (Calculation) If a table has 100 rows and you execute `SELECT DISTINCT country FROM Users;`, and 40 users are from 'USA', 40 are from 'UK', and 20 are from 'Germany', how many rows are in the result?

A) 100

B) 20

C) 3

D) 1

> The `DISTINCT` keyword removes all duplicate values. Since there are only 3 unique strings in the `country` column ('USA', 'UK', 'Germany'), only 3 rows are returned.

Q19. (Code-Tracing) What does the following command achieve?

mysql-terminal

```
ALTER TABLE Students DROP COLUMN age;
```

A) Deletes the rows for students who have an age recorded.

B) Sets the age of all students to NULL.

C) Physically removes the 'age' attribute from the table structure.

D) Renames the 'age' column to 'DROP'.

> `ALTER TABLE ... DROP COLUMN` is a DDL command that modifies the schema by permanently deleting a column and all data stored within it.

Q20. (Conceptual) Which of the following is a key requirement for the `UNION` operator to work between two `SELECT` statements?

A) Both tables must have the same Primary Key.

B) Both queries must return the same number of columns with compatible data types in the same order.

C) The first table must have more rows than the second.

D) A `WHERE` clause must be present in both queries.

> For `UNION` to be valid, the two result sets must be "Union Compatible." This means the number of columns and their corresponding data types must match so the rows can be merged into a single set.

Deep Dive: SQL Execution Order vs. Syntax Order

One of the most confusing parts of SQL for developers is that the engine executes clauses in a completely different order than you write them. While you write `SELECT` first, it is actually one of the last things the engine processes.

- **1. FROM & JOIN:** The engine first identifies the source tables and performs any joins to create a "working set" of data.
- **2. WHERE:** The engine filters the working set based on the row-level conditions.
- **3. GROUP BY:** The remaining rows are grouped together based on shared values in the specified columns.
- **4. HAVING:** The aggregate groups are filtered (e.g., "only show groups where `SUM(sales) > 1000`").
- **5. SELECT:** The engine finally decides which columns to project. This is where `DISTINCT` and `AS` aliases are applied.
- **6. ORDER BY:** The final result set is sorted.

Why this matters: This is why you cannot use a column alias defined in the `SELECT` clause inside a `WHERE` clause (because the WHERE happens before the SELECT), but you *can* use it in the `ORDER BY` clause.

Stable Web Resources

- Wikipedia: SQL
- W3Schools: SQL Tutorial

Quick Revision Cheat Sheet

- **DDL (Structure):** CREATE, ALTER, DROP, TRUNCATE.
- **DML (Data):** INSERT, UPDATE, DELETE.
- **DQL (Query):** SELECT.
- **Precedence:** Parentheses → AND → OR.
- **NULL Rule:** Use `IS NULL`, never `= NULL`.
- **Wildcards:** `%` (many), _ (one).
- **Sorting:** `DESC` for highest first; default is `ASC`.

END OF SUBTOPIC 06 — MOVING TO SUBTOPIC 07

Subtopic 07: Advanced SQL (20 MCQs)

Quick Notes

- **Joins:**
  - *INNER JOIN:* Returns rows with matches in both tables.
  - *LEFT/RIGHT JOIN:* Returns all rows from the primary table and matches from the secondary (filling with NULLs where no match exists).
  - *FULL OUTER JOIN:* Returns all rows from both tables, matching where possible.
  - *SELF JOIN:* Joining a table with itself (requires aliases).
- **Subqueries:** Queries nested inside another query. **Correlated Subqueries** reference columns from the outer query and execute once for every row processed by the outer query.
- **Common Table Expressions (CTEs):** Defined using the `WITH` clause. They improve readability and can be recursive.
- **Window Functions:** Perform calculations across a set of table rows related to the current row (e.g., `ROW_NUMBER()`, `RANK()`, `LEAD()`, `LAG()`). They do not collapse rows like `GROUP BY`.
- **Set Operators:** `UNION` (distinct), `UNION ALL` (includes duplicates), `INTERSECT` (common), `EXCEPT/MINUS` (difference).

Common Traps & Confusions

- **JOIN vs. WHERE Filtering:** Putting a filter in the `ON` clause of a `LEFT JOIN` behaves differently than putting it in the `WHERE` clause. Filtering in `WHERE` can accidentally turn a `LEFT JOIN` into an `INNER JOIN`.
- **UNION vs. UNION ALL:** `UNION` performs a costly deduplication step. Use `UNION ALL` if you know the sets are disjoint or if you specifically want duplicates for performance reasons.
- **RANK() vs. DENSE_RANK():** `RANK()` leaves gaps in numbering after ties (1, 2, 2, 4). `DENSE_RANK()` does not leave gaps (1, 2, 2, 3).

- **IN vs. EXISTS:** `EXISTS` is generally more efficient for subqueries because it stops searching as soon as it finds a single match (Short-circuiting).

MCQs

Q1. (Code-Tracing) Analyze the following tables and the query below. What will be the result of the query?

sql-join-challenge

```sql
-- Table: Users (id, name)
-- (1, 'Alice'), (2, 'Bob')

-- Table: Profiles (u_id, city)
-- (1, 'NY'), (3, 'LA')

SELECT u.name, p.city
FROM Users u
LEFT JOIN Profiles p ON u.id = p.u_id;
```

A) ('Alice', 'NY')

B) ('Alice', 'NY'), ('Bob', 'LA')

C) ('Alice', 'NY'), ('Bob', NULL)

D) ('Alice', 'NY'), (NULL, 'LA')

> A `LEFT JOIN` keeps all rows from the left table (Users). Alice has a match (NY). Bob has no match in Profiles, so his `city` is returned as NULL. The row for 'LA' in Profiles is ignored because it has no match in the left table.

Q2. (Scenario-Based) You have an `Employees` table with columns `emp_id`, `name`, and `manager_id`. Which SQL technique is required to generate a list of all employees alongside the name of their direct manager in a single result set?

A) Full Outer Join

B) Cross Join

C) Self-Join

D) Natural Join

> To link an employee to their manager (who is also an employee in the same table), you must join the table with itself. You aliasing the table twice (e.g., `FROM Employees e JOIN Employees m ON e.manager_id = m.emp_id`) to distinguish between the "employee" role and the "manager" role.

Q3. (Conceptual) What defines a **Correlated Subquery** in SQL?

A) It is a subquery that executes once and stores the result in a temporary table.

B) It is a subquery that depends on values from the outer query, executing once for every row processed by the outer query.

C) It is a subquery that uses the `UNION` operator.

D) It is a subquery placed specifically in the `FROM` clause.

> A correlated subquery refers to columns in the outer query's tables. Because its results depend on the specific row being processed by the outer query, the engine must re-evaluate the subquery for each row, which can impact performance.

Q4. (Code-Tracing) What is the purpose of the `WITH` clause in the following SQL snippet?

postgres - cte_logic

```sql
WITH RegionalSales AS (
    SELECT region, SUM(amount) AS total
    FROM Orders GROUP BY region
)
SELECT region FROM RegionalSales
WHERE total > (SELECT AVG(total) FROM RegionalSales);
```

A) It creates a permanent view in the database.

B) It acts as a temporary index to speed up the query.

C) It defines a Common Table Expression (CTE) to simplify complex logic and allow re-use of a subquery result.

D) It filters the rows before the grouping occurs.

> The `WITH` clause defines a CTE named `RegionalSales`. This allows the developer to treat the aggregated result like a temporary table within the scope of a single query, making it easier to reference the result multiple times (as seen in the `SELECT` and the subquery).

Q5. (Calculation) Table A has 5 rows and Table B has 10 rows. If you perform a **CROSS JOIN** between A and B, but the `WHERE` clause contains a condition that is always false (e.g., `WHERE 1=0`), how many rows will be returned?

A) 50

B) 15

C) 5

D) 0

> A CROSS JOIN (Cartesian Product) initially produces 5 * 10 = 50 rows. However, the `WHERE` clause is applied to the result set. If the condition is always false, all 50 rows are filtered out, resulting in an empty set.

Q6. (Scenario-Based) A bank wants a list of customers who have a "Savings Account" but **do not** have a "Credit Card". Both accounts are stored in the `Accounts(cust_id, type)` table. Which set operator is most appropriate?

A) UNION

B) INTERSECT

C) EXCEPT (or MINUS)

D) UNION ALL

> The `EXCEPT` operator returns all distinct rows from the first query that are not present in the second query. By subtracting the set of 'Credit Card' owners from the set of 'Savings' owners, we get the desired list.

Q7. (Code-Tracing) Examine the window function below. If three employees have the exact same salary and are tied for the highest pay, what will the `RANK()` function return for them?

sql-window-functions

```sql
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) as pos
FROM Employees;
```

A) 1, 2, 3

B) 1, 1, 1 (and the next employee will be 4)

C) 1, 1, 1 (and the next employee will be 2)

D) NULL

> `RANK()` assigns the same rank to ties. However, it leaves a "gap" in the sequence. If three people are ranked 1, the 4th person receives rank 4. (If `DENSE_RANK()` were used, the 4th person would be rank 2).

Q8. (Comparison) What is the primary operational difference between a standard **VIEW** and a **MATERIALIZED VIEW**?

A) A standard View can be indexed, but a Materialized View cannot.

B) A standard View stores data on disk, while a Materialized View is strictly in RAM.

C) A standard View is a virtual query executed at runtime; a Materialized View stores the query result physically on disk for faster access.

D) Standard Views are DDL, while Materialized Views are DML.

> Standard Views are just "saved queries" that run every time they are called. Materialized Views save the data result of the query. This makes them much faster for heavy aggregations, though they must be "refreshed" when the underlying data changes.

Q9. (Calculation) Table `T1` has 3 rows with values {1, 2, NULL}. Table `T2` has 3 rows with {1, NULL, NULL}. How many rows are returned by an **INNER JOIN** on these tables using `T1.val = T2.val`?

A) 3

B) 2

C) 1

D) 0

> In SQL joins, `NULL = NULL` is **Unknown**, not True. Therefore, NULL values never match each other in a join. Only the value `1` matches between the two tables, resulting in exactly one row.

Q10. (Code-Tracing) What does the `EXISTS` keyword do in the following query?

mysql - subquery_test

```
SELECT name FROM Departments d
WHERE EXISTS (
    SELECT 1 FROM Employees e
    WHERE e.dept_id = d.id AND e.salary > 10000
```

```
);
```

A) Returns all employees earning more than 10,000.

B) Returns departments and the sum of their high-earning employees.

C) Returns the names of departments that have at least one employee earning more than 10,000.

D) Returns an error because the subquery selects '1' instead of a column.

> `EXISTS` returns true if the subquery returns *any* rows. It doesn't matter what columns are selected (which is why `SELECT 1` is common). It effectively finds departments that meet the criteria.

Q11. (Scenario-Based) Which of the following is the most standard SQL approach to find the **second highest** salary in an `Employees` table without using window functions?

A) SELECT MAX(salary) FROM Employees WHERE salary < MAX(salary);

B) SELECT MAX(salary) FROM Employees WHERE salary < (SELECT MAX(salary) FROM Employees);

C) SELECT salary FROM Employees ORDER BY salary DESC LIMIT 1, 1;

D) SELECT TOP 2 salary FROM Employees ORDER BY salary DESC;

> Option B is standard SQL. It first finds the absolute maximum salary in a subquery, then finds the maximum salary from the set of salaries that are strictly less than that absolute maximum. (Option C is MySQL specific).

Q12. (Comparison) When should a developer prefer **UNION ALL** over **UNION**?

A) When they need to ensure the final result set contains no duplicates.

B) When the developer knows the two sets are already disjoint or when performance is more important than removing duplicates.

C) When the column names in the two tables are different.

D) When the query contains a `GROUP BY` clause.

> `UNION` performs a sort and unique operation to remove duplicates, which is CPU and memory intensive. `UNION ALL` simply appends the two sets, making it significantly faster if deduplication isn't required.

Q13. (Code-Tracing) Analyze the use of the `CASE` expression. What is returned for a product with `stock = 0`?

sql-logic - case_expression

```sql
SELECT name,
  CASE
    WHEN stock > 10 THEN 'In Stock'
    WHEN stock > 0 THEN 'Low Stock'
    ELSE 'Out of Stock'
  END as availability
FROM Products;
```

A) Low Stock

B) In Stock

C) Out of Stock

D) NULL

> SQL evaluates `CASE` conditions in order. Since 0 is not > 10 and not > 0, it falls through to the `ELSE` block, returning 'Out of Stock'.

Q14. (Conceptual) What is a **Non-Equijoin**?

A) A join that results in an empty set.

B) A join performed on two tables with no common columns.

C) A join that uses comparison operators other than equality (e.g., <, >, BETWEEN).

D) A join that combines more than three tables.

> Most joins are "Equijoins" (using =). A Non-equijoin uses other logic, such as joining an employee to a `Salary_Grades` table where the salary is `BETWEEN` the low and high bounds of the grade.

Q15. (Scenario-Based) You have a table with millions of duplicate rows due to a bug. You want to delete duplicates but keep exactly **one** instance of each record. Which approach is most efficient?

A) DELETE FROM Table WHERE id NOT IN (SELECT id FROM Table);

B) Use a CTE with `ROW_NUMBER()` partitioned by all data columns, and delete where the row number > 1.

C) Use a `CROSS JOIN` to find duplicates and delete them.

D) DROP the table and recreate it manually.

> Using `ROW_NUMBER() OVER(PARTITION BY [all columns])` identifies duplicates by assigning '1' to the first instance and '2, 3...' to duplicates. You can then wrap this in a CTE and delete any row where the assigned number is greater than 1.

Q16. (Code-Tracing) What is the result of the `HAVING` clause in this query?

sql-aggregation-filter

```sql
SELECT dept_id, AVG(salary)
FROM Employees
GROUP BY dept_id
HAVING COUNT(*) > 5;
```

A) It returns all employees in departments with more than 5 people.

B) It returns only the departments where every employee earns more than the average.

C) It returns the average salary only for those departments that employ more than five people.

D) It returns an error because `COUNT(*)` is not in the `SELECT` list.

> `HAVING` filters groups after the `GROUP BY` has been processed. Here, it calculates the count of each department group and only allows the department to appear in the final output if its size is > 5.

Q17. (Calculation) If Relation R(A, B, C) and Relation S(C, D, E) are joined using a **NATURAL JOIN**, what will be the attributes (Degree) of the resulting relation?

A) {A, B, C, C, D, E} (Degree 6)

B) {A, B, C, D, E} (Degree 5)

C) {C} (Degree 1)

D) {A, B, D, E} (Degree 4)

> A Natural Join automatically joins on columns with the same name (C) and removes the duplicate column from the output. The resulting attributes are the union of the two sets: A, B, C, D, and E.

Q18. (Conceptual) What is a **Scalar Subquery**?

A) A subquery that returns a full table.

B) A subquery that returns a single column but many rows.

C) A subquery that returns exactly one value (one row and one column).

D) A subquery that uses mathematical scale functions.

> A scalar subquery is a query that returns a single value. Because it returns only one value, it can be used in places where a single constant is expected, such as in the `SELECT` list or as a comparison in a `WHERE` clause.

Q19. (Scenario-Based) An e-commerce company wants to keep an audit trail. Every time a price is updated in the `Products` table, the old price and the time of change must be saved into a `Price_History` table automatically. Which SQL object is designed for this?

A) Stored Procedure

B) Constraint

C) Trigger

D) Index

> A Trigger is a special type of stored procedure that automatically runs ("fires") when a specific event occurs in the database, such as an `UPDATE`, `INSERT`, or `DELETE`.

Q20. (Code-Tracing) Trace the logic of this three-table join. Which customers will appear in the output?

sql-triple-join

```sql
SELECT c.name
FROM Customers c
JOIN Orders o ON c.id = o.cust_id
JOIN OrderItems i ON o.id = i.order_id
WHERE i.product_name = 'Laptop';
```

A) All customers who have ever placed an order.

B) All customers who have ever viewed a Laptop.

C) Customers who have placed at least one order containing a 'Laptop'.

D) Customers who have only ordered Laptops and nothing else.

> The query joins Customers to Orders, and then Orders to OrderItems. The `WHERE` clause filters the result to only those rows where the item is a 'Laptop'. Thus, only customers who purchased a laptop are returned.

Deep Dive: Join Algorithms & Optimization

When you write a `JOIN`, the Database Engine chooses between three primary physical algorithms to execute it:

- **Nested Loop Join:** For every row in Table A, it scans Table B. Excellent for small tables or when the join column is indexed.
- **Hash Join:** The engine creates a hash table in memory for the smaller table and then probes it with the larger table. Very fast for large, unsorted datasets.
- **Sort-Merge Join:** Both tables are sorted by the join key, and then the engine "zips" them together. Most efficient when both tables are already sorted or have clustered indexes on the join column.

The "SARGable" rule: To ensure joins and filters are fast, queries must be "Search ARGumentable." If you use a function on a join column (e.g., `ON UPPER(a.name) = b.name`), the database cannot use an index on `a.name`, forcing a slow full table scan. Always try to keep columns "naked" in join conditions.

Stable Web Resources

- Wikipedia: SQL Joins
- Use The Index, Luke (SQL Performance Tuning)

Quick Revision Cheat Sheet

- **Inner Join:** Intersection. **Left Join:** Left side + NULLs for missing right.
- **Self Join:** Table joined to itself (Aliases mandatory).
- **EXISTS:** Returns True/False, stops early (Efficient).
- **CTEs:** Readable temporary result sets (`WITH`).
- **RANK():** Gaps in ties. **DENSE_RANK():** No gaps in ties.
- **UNION ALL:** Faster than UNION (no deduplication).

- **NULL in Joins:** NULL never equals NULL.

END OF SUBTOPIC 07 — MOVING TO SUBTOPIC 08

Subtopic 08: Database Design & Normalization (20 MCQs)

Quick Notes

- **Functional Dependency (FD):** X → Y means if two tuples have the same X value, they must have the same Y value.
- **1NF (First Normal Form):** All attribute values must be atomic (no multi-valued or composite attributes).
- **2NF (Second Normal Form):** Must be in 1NF and have **No Partial Dependencies** (every non-prime attribute must depend on the whole primary key, not just part of it).
- **3NF (Third Normal Form):** Must be in 2NF and have **No Transitive Dependencies** (non-prime attributes should not depend on other non-prime attributes).
- **BCNF (Boyce-Codd Normal Form):** A stricter version of 3NF. For every FD X → Y, X must be a super key.
- **Decomposition:** Splitting a table to remove anomalies. It must be **Lossless-Join** (you can reconstruct the original table) and ideally **Dependency Preserving**.

Common Traps & Confusions

- **2NF vs 3NF:** Partial dependencies (2NF) only exist when you have a *composite* primary key. If your PK is a single column, the table is automatically in 2NF.
- **Prime vs Non-Prime:** A prime attribute is part of *any* candidate key. A non-prime attribute is part of *no* candidate key.
- **Over-Normalization:** While higher normal forms reduce redundancy, they increase the number of JOINs required, which can degrade read performance in high-traffic systems.

- **Lossy Decomposition:** If you split a table incorrectly, joining them back might create "spurious tuples" (extra rows that weren't in the original data), making the data invalid.

MCQs

Q1. (Scenario-Based) A school database has a table `Student_Courses(student_id, student_name, course_id, course_fee)`. If the school increases the fee for 'CS101', the admin must update every single row where a student is enrolled in 'CS101'. Failing to update one row creates a data conflict. What is this problem called?

A) Insertion Anomaly

B) Deletion Anomaly

C) Update Anomaly

D) Referential Integrity Violation

> An Update Anomaly occurs when data is redundant, and changing a value in one place requires changes in multiple rows. If the fee were stored in a separate `Courses` table linked by `course_id`, only one update would be needed.

Q2. (Code-Tracing) Examine the following SQL table definition. Why does this table violate **1NF (First Normal Form)**?

sql-schema-audit

```sql
CREATE TABLE Developer (
    dev_id INT PRIMARY KEY,
    name VARCHAR(50),
    skills VARCHAR(255) -- Stores 'Java, Python, SQL'
);
```

A) Because it has no foreign keys.

B) Because the `name` column might have duplicates.

C) Because the `skills` column contains non-atomic, multi-valued data.

D) Because `VARCHAR(255)` is too large for a skill list.

> 1NF requires all attributes to be atomic. Storing a comma-separated list of values (like 'Java, Python') in a single cell violates 1NF. The correct approach is a separate `Dev_Skills` table where each skill is its own row.

Q3. (Calculation) A relation R(A, B, C, D) has the following Functional Dependencies:
{A → B, B → C, C → D}
What is the **closure** of attribute A, denoted as {A}+?

A) {A, B}

B) {A, B, C}

C) {A, B, C, D}

D) {A}

> Attribute closure is the set of all attributes that can be determined by a given set.
> 1. Start with {A}.
> 2. A → B, so {A, B}.
> 3. B → C, so {A, B, C}.
> 4. C → D, so {A, B, C, D}.

Q4. (Conceptual) In a table with a composite primary key `(OrderID, ProductID)`, a non-prime attribute `OrderDate` depends only on `OrderID`. Which normal form is being violated?

A) 1NF

B) 2NF

C) 3NF

D) BCNF

> This is a **Partial Dependency** because `OrderDate` is determined by only a part of the composite primary key (OrderID). To satisfy 2NF, every non-prime attribute must depend on the full candidate key.

Q5. (Scenario-Based) You have a `Staff` table: `(staff_id, name, dept_id, dept_manager_id)`. The Primary Key is `staff_id`.
The dependencies are: `staff_id → dept_id` and `dept_id → dept_manager_id`.
What is the highest normal form this table satisfies?

A) 1NF

B) 2NF

C) 3NF

D) BCNF

> The table is in 2NF because the PK is a single column (no partial dependencies). However, it violates 3NF because `staff_id → dept_id → dept_manager_id` is a **Transitive Dependency**. `dept_manager_id` (non-prime) depends on `dept_id` (non-prime).

Q6. (Code-Tracing) A senior architect suggests "Denormalizing" a high-traffic analytics table. What is the most likely reason for this recommendation?

A) To improve data integrity and reduce redundancy.

B) To reduce the amount of physical disk space used.

C) To improve read performance by reducing the number of costly JOIN operations.

D) To make the database schema easier for junior developers to understand.

> Denormalization is the intentional introduction of redundancy. In high-read, low-write environments (like data warehouses), denormalizing tables into a flat structure speeds up SELECT queries because the engine doesn't have to merge multiple tables.

Q7. (Comparison) What is the key difference between **3NF** and **BCNF**?

A) 3NF allows partial dependencies, while BCNF does not.

B) BCNF is used only for NoSQL databases.

C) 3NF allows a non-prime attribute to determine a prime attribute, whereas BCNF requires every determinant to be a super key.

D) BCNF allows transitive dependencies, while 3NF does not.

> BCNF is more restrictive than 3NF. In 3NF, the FD $X \rightarrow A$ is allowed if A is a prime attribute. In BCNF, X MUST be a super key, regardless of whether A is prime or not.

Q8. (Calculation) Given the relation R(A, B, C) and FDs: **{A → B, B → A, B → C}**. Which of the following is a **Candidate Key**?

A) Only {A}

B) Only {B}

C) Both {A} and {B}

D) {A, B}

> {A}+ = {A, B, C} and {B}+ = {B, A, C}. Since both A and B can uniquely identify all attributes in the relation, they are both candidate keys. {A, B} is a super key but not a candidate key because it isn't minimal.

Q9. (Conceptual) A decomposition of a relation R into R1 and R2 is said to be **Lossless-Join** if:

A) R1 ∩ R2 = empty set.

B) R1 ∪ R2 = R.

C) R1 ∩ R2 is a candidate key for either R1 or R2.

D) The number of rows in R1 plus R2 equals the rows in R.

> Lossless-join decomposition ensures that when you join R1 and R2 back together, you get exactly the original relation R. This is mathematically guaranteed if the common attributes (the intersection) form a candidate key in at least one of the decomposed tables.

Q10. (Scenario-Based) In an `Orders` table, if deleting a single customer's last order also results in the database losing all of that customer's contact information (because it was only stored in the order record), what anomaly has occurred?

A) Insertion Anomaly

B) Deletion Anomaly

C) Update Anomaly

D) Integrity Anomaly

> A Deletion Anomaly occurs when the deletion of data representing one real-world fact (an order) unintentionally results in the loss of data representing a completely different fact (customer contact info) because the table is poorly designed.

Q11. (Calculation) A relation R(P, Q, R, S, T) has FDs: **{P → Q, RS → T, Q → S}**. What is the closure **{P, R}+**?

A) {P, R, Q}

B) {P, R, Q, S}

C) {P, R, Q, S, T}

D) {P, R}

> 1. Start with {P, R}.
> 2. P → Q, so {P, R, Q}.

> 3. Q → S, so {P, R, Q, S}.
> 4. RS → T (we have both R and S), so {P, R, Q, S, T}.

Q12. (Code-Tracing) You are auditing a database and find the following schema. Which normal form does it currently violate?

sql-schema-check

```sql
CREATE TABLE Inventory (
    item_id INT,
    warehouse_id INT,
    item_name VARCHAR(50),
    warehouse_location VARCHAR(100),
    PRIMARY KEY (item_id, warehouse_id)
);
```

A) 1NF

B) 2NF

C) 3NF

D) BCNF

> This violates 2NF because of partial dependencies. `item_name` depends only on `item_id`, and `warehouse_location` depends only on `warehouse_id`. Both non-prime attributes depend on only *part* of the composite primary key.

Q13. (Comparison) Why is **Dependency Preservation** an important goal during normalization?

A) It ensures that the database uses less disk space.

B) It allows all integrity constraints to be checked within individual tables without performing costly JOINs.

C) It prevents NULL values from being inserted into foreign keys.

D) It is required for a decomposition to be lossless.

> If a decomposition preserves dependencies, any functional dependency of the original relation can be checked by looking at just one of the resulting relations. If not preserved, checking a constraint would require joining multiple tables, which is very slow for every INSERT/UPDATE.

Q14. (Scenario-Based) A database currently in 3NF has two candidate keys: `(StudentID, CourseID)` and `(StudentID, Email)`. An FD exists: `Email → CourseID`. Does this table satisfy BCNF?

A) Yes, because it is in 3NF.

B) No, because Email is a determinant but not a super key.

C) Yes, because Email is a prime attribute.

D) No, because of transitive dependency.

> In BCNF, every determinant must be a super key. In the FD `Email → CourseID`, Email is the determinant. Since `Email` alone cannot uniquely identify a row (a student can have many courses), it is not a super key. Therefore, the table violates BCNF.

Q15. (Calculation) How many candidate keys exist for R(A, B, C) if the FDs are **{A → B, B → C, C → A}**?

A) 1

B) 2

C) 3

D) 0

> {A}+ = {A, B, C}. {B}+ = {B, C, A}. {C}+ = {C, A, B}. Since each single attribute can determine the entire relation, A, B, and C are each separate candidate keys.

Q16. (Conceptual) What is a **Multi-valued Dependency (MVD)**, which is the focus of 4NF?

A) When one attribute determines a non-prime attribute.

B) When two attributes are identical.

C) When one attribute determines a set of values for another attribute, and these values are independent of other attributes in the table.

D) When a primary key consists of more than five columns.

> MVD (X ↠ Y) occurs when a value of X is associated with a set of values for Y, and this set is independent of any other attributes. 4NF removes these dependencies to prevent redundant rows.

Q17. (Scenario-Based) A developer refuses to normalize a table because they claim "the data is small and JOINs are slow." Which risk are they primarily ignoring?

A) Disk space exhaustion.

B) SQL syntax complexity.

C) Data inconsistency and anomalies during future data entry.

D) Hardware compatibility.

> While performance is important, normalization's primary purpose is to maintain data integrity. Unnormalized tables lead to anomalies where data can become contradictory (inconsistent) over time.

Q18. (Calculation) If a relation is in **BCNF**, which of the following is also guaranteed to be TRUE?

A) It is in 4NF.

B) It is in 3NF.

C) It has no composite keys.

D) It has only one candidate key.

> The normal forms are a hierarchy. BCNF is stricter than 3NF. Therefore, any relation that satisfies BCNF automatically satisfies 3NF, 2NF, and 1NF.

Q19. (Conceptual) In database normalization theory, what is the **"Minimal Cover"** (or Canonical Cover)?

A) The smallest possible primary key for a table.

B) A decomposition that uses the fewest number of tables.

C) A simplified set of functional dependencies that is equivalent to the original set but has no redundant dependencies or attributes.

D) The smallest set of rows that represents the entire table.

> The Minimal Cover is used during the normalization process to ensure that we are working with the simplest possible representation of data relationships, removing any FDs that can be inferred from others.

Q20. (Code-Tracing) You are tasked with fixing a 2NF violation in a schema. What is the standard SQL procedure to achieve this?

A) Add a `UNIQUE` constraint to every column.

B) Use `CREATE TABLE` to move the partially dependent attributes to a new table where the determinant becomes the Primary Key.

C) Combine all columns into a single `TEXT` field to satisfy atomicity.

D) Delete the composite key and replace it with an `AUTO_INCREMENT` ID.

> Normalizing involves decomposition. To fix a 2NF partial dependency (A, B) → C where A → C, you create a new table for (A, C) where A is the PK, and remove C from the original table.

Deep Dive: Lossless-Join & Spurious Tuples

The **Lossless-Join property** is the most critical requirement for decomposition. If a relation R is decomposed into R1 and R2, the decomposition is lossless if and only if R = `R1 ⋈ R2`.

If the intersection of R1 and R2 is NOT a key for either, the join will produce **Spurious Tuples**. These are rows that were not in the original database. For example, if you split a table of Employees and Departments by only using "Location" as a link, and two different departments are in the same location, the join will incorrectly associate every employee in that location with both departments. Normalization prevents this by ensuring tables are only linked via Candidate Keys (Foreign Key → Primary Key relationships).

Stable Web Resources

- Wikipedia: Database Normalization
- GeeksforGeeks: Normal Forms

Quick Revision Cheat Sheet

- **1NF:** Atomicity (No lists in cells).
- **2NF:** No Partial Dependencies (Whole PK or nothing).
- **3NF:** No Transitive Dependencies (Non-prime → Non-prime).
- **BCNF:** Every determinant must be a Super Key.
- **Anomalies:** Insertion (cannot add), Deletion (unintended loss), Update (redundant changes).
- **Lossless Join:** Reconstruct original data perfectly.

- **Dependency Preservation:** Check constraints without JOINs.

END OF SUBTOPIC 08 — MOVING TO SUBTOPIC 09

Subtopic 09: Transaction Management (15 MCQs)

Quick Notes

- **Transaction:** A single logical unit of work that accesses and possibly modifies the contents of a database.
- **ACID Properties:**
  - *Atomicity:* All operations of the transaction are reflected in the database, or none are (All or Nothing).
  - *Consistency:* Execution of a transaction in isolation preserves the consistency of the database (Valid state to Valid state).
  - *Isolation:* Transactions are hidden from each other until they are finished (Concurrency control).
  - *Durability:* Once a transaction is committed, changes persist even in the event of a system failure.
- **Transaction States:** Active → Partially Committed → Committed. Or Active → Failed → Aborted.
- **TCL Commands:** `COMMIT` (save), `ROLLBACK` (undo), `SAVEPOINT` (partial rollback point).

Common Traps & Confusions

- **Partially Committed vs. Committed:** A transaction is "Partially Committed" when the final statement is executed, but the logs are still in the volatile buffer. It is only "Committed" once the log records are safely flushed to non-volatile storage (disk).

- **Consistency vs. Integrity:** Consistency is the responsibility of the programmer (logical correctness), while Integrity is enforced by the DBMS (constraints like FK/PK).
- **Read-Only Transactions:** Even if a transaction only performs `SELECT`, it still follows ACID properties and manages locks to ensure a consistent view of data.

MCQs

Q1. (Scenario-Based) During a high-volume sale, a customer completes a purchase. The inventory is decremented, and the payment is processed. However, before the shipping label is generated, the database server loses power. Upon reboot, the system reverts the inventory and payment records to their state before the purchase began. Which ACID property is being enforced?

A) Consistency

B) Isolation

C) Atomicity

D) Durability

> Atomicity ensures "All or Nothing." Since the entire purchase "unit of work" was not completed (the shipping label wasn't logged), the DBMS rolls back all partial changes to ensure no partial data exists.

Q2. (Code-Tracing) Analyze the following transaction block. What will be the final value of `balance` in the database for `id = 1`?

sql-transaction-trace

```sql
-- Initial Balance: 1000
START TRANSACTION;
UPDATE accounts SET balance = balance - 200 WHERE id = 1;
SAVEPOINT sp1;
UPDATE accounts SET balance = balance - 300 WHERE id = 1;
ROLLBACK TO sp1;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
COMMIT;
```

A) 500

B) 700

C) 400

D) 1000

> 1. Start: 1000.
> 2. Subtract 200: 800.
> 3. Savepoint sp1.
> 4. Subtract 300: 500.
> 5. Rollback to sp1: Reverts to 800.
> 6. Subtract 100: 700.
> 7. Commit: 700 is saved.

Q3. (Conceptual) In the standard transaction state diagram, what is the critical difference between the **Partially Committed** state and the **Committed** state?

A) Partially Committed means only 50% of SQL statements have run.

B) Committed state is only reached after all log records have been physically written to non-volatile storage.

C) Partially Committed transactions can no longer be rolled back.

D) There is no difference; they are synonymous.

> A transaction is "Partially Committed" when the code finishes. It is "Committed" only when the DBMS confirms that the recovery logs are safely on the disk. If a crash happens while "Partially Committed," the transaction must be rolled back.

Q4. (Scenario-Based) A developer writes a transaction that transfers 500 from Account A to Account B. If the developer accidentally writes code that adds 500 to Account B but only subtracts 400 from Account A, the database will finish in an invalid state. This is a violation of which property?

A) Atomicity

B) Consistency

C) Isolation

D) Durability

> Consistency is a "semantic" property. The total sum of money in the bank must remain constant. If the logic of the transaction is flawed, the DBMS cannot always detect it (unless constraints are set); it is the programmer's responsibility to ensure the transaction moves the DB from one valid state to another.

Q5. (Code-Tracing) What is the result of the following `psql` session?

postgres-session

```
BEGIN;
INSERT INTO logs VALUES ('Action 1');
DELETE FROM logs WHERE msg = 'Action 1';
ROLLBACK;
SELECT COUNT(*) FROM logs;
```

A) 1

B) 0 (assuming table was empty initially)

C) Error: Cannot delete what was just inserted.

D) NULL

> A `ROLLBACK` undoes every operation performed since the `BEGIN` command. Even though an insert and delete happened, the rollback wipes the slate clean as if the transaction never started.

Q6. (Comparison) How does **Durability** typically handle data that is currently in the DB's RAM buffer but has not yet been written to the data file when a crash occurs?

A) The data is lost forever.

B) The database engine prevents crashes until the buffer is clear.

C) The engine uses a Write-Ahead Log (WAL) on disk to "Redo" the changes upon restart.

D) The OS automatically saves the RAM to disk during power failure.

> Durability is achieved by the Log Manager. Before any data is updated in the "lazy" buffer, a record of the change is written sequentially to a log file on disk. On restart, the DBMS scans this log to re-apply any committed changes that didn't make it to the main data files.

Q7. (Conceptual) A transaction that cannot be completed successfully due to an internal error (like a deadlock or a hardware read failure) enters which state?

A) Partially Committed

B) Aborted

C) Failed

D) Terminated

> A transaction enters the **Failed** state as soon as it is discovered that normal execution can no longer proceed. Once the DBMS finishes the rollback of partial changes, the state changes to **Aborted**.

Q8. (Scenario-Based) In a multi-user environment, User A is calculating the sum of all account balances. Simultaneously, User B transfers 100 from one account to another. User A's report must not see User B's partial work (e.g., seeing one account deducted but not yet the other credited). This is a requirement for:

A) Durability

B) Consistency

C) Isolation

D) Atomicity

> Isolation ensures that concurrent transactions do not see each other's intermediate or uncommitted states. This prevents "Dirty Reads" and ensures data looks consistent to every user.

Q9. (Calculation/Comparison) Consider the "Log Sequence" below for an "Undo/Redo" recovery strategy. If a Checkpoint occurred at Time T, and the system crashed at T+1, which transactions must be **Redone**?

recovery-log-viewer

```
T0: [Start T1]
T1: [T1 Update A]
T2: [Checkpoint]
T3: [Start T2]
T4: [T1 Commit]
T5: [T2 Update B]
-- CRASH --
```

A) T1 and T2

B) Only T1

C) Only T2

D) Neither

> T1 committed before the crash, so its changes must be permanent (Redo). T2 was active but did not commit, so its changes must be undone (Undo). Checkpoints help the DBMS know

> that everything before T2 was definitely on disk, but committed transactions since the checkpoint still need re-applying.

Q10. (Code-Tracing) In most RDBMS (like MySQL/PostgreSQL), what is the default behavior of a single SQL statement (e.g., `UPDATE x SET y=1;`) not wrapped in an explicit transaction block?

A) It is rejected.

B) It stays in the "Active" state until the user logs out.

C) It is treated as an "Autocommit" transaction (Implicit Transaction).

D) It is only saved if the user types 'COMMIT' manually later.

> Standard SQL operates in "Autocommit" mode by default. Every single statement is wrapped in its own implicit transaction that commits immediately upon successful execution.

Q11. (Conceptual) Which database component is primarily responsible for ensuring the **Atomicity** and **Durability** of transactions?

A) Query Optimizer

B) Recovery Manager (Log Manager)

C) Concurrency Controller

D) Schema Compiler

> The Recovery Manager uses logs (Undo logs for Atomicity and Redo logs for Durability) to ensure that transactions are either fully completed or fully undone, even across system crashes.

Q12. (Comparison) What is the advantage of using **Shadow Paging** over Log-based recovery?

A) It is much faster for high-concurrency systems.

B) It eliminates the overhead of writing to a log file by maintaining two versions of the database pages.

C) It is more space-efficient on disk.

D) It is the standard used by Oracle and SQL Server.

> Shadow Paging maintains a "Current" page table and a "Shadow" page table. Changes are made to a copy; if the transaction commits, the pointer switches to the copy. It avoids log processing but suffers from data fragmentation and is rarely used in modern massive RDBMS.

Q13. (Scenario-Based) An application requires that if a transaction is committed on a database in the USA, a synchronized backup database in Singapore must also have that data permanently stored before the transaction is acknowledged as "Success" to the user. This is an example of:

A) Local Durability

B) Synchronous Distributed Durability

C) Isolation Level 4

D) Lazy Writing

> Distributed transactions must ensure Durability across all nodes. Synchronous durability means the "Commit" doesn't return to the user until all replicas have confirmed the write to their respective non-volatile storages.

Q14. (Calculation/Comparison) Given the following transaction sequence, which statement describes the **Durability** requirement?

```
1. User clicks 'Submit Payment'
2. DB updates 'Status' to 'Paid'
3. DB sends 'Success' to UI
```

A) Status can be 'Paid' even if the server crashes before step 3.

B) Once step 3 is reached, the 'Paid' status must be recoverable even if the server loses power a millisecond later.

C) Step 2 must be visible to all other users immediately.

D) The user can undo the payment by clicking 'Back' in the browser.

> Durability dictates that once the system acknowledges a success (Step 3), the data change is permanent. The DB must have flushed the log to disk *before* it sent that success message.

Q15. (Conceptual) If a database is in a state where it satisfies all defined **Integrity Constraints** (Primary Keys, Foreign Keys, Check Constraints), it is said to have:

A) Atomicity

B) Consistency

C) Isolation

D) Persistence

> The 'C' in ACID stands for Consistency. A transaction must leave the database in a consistent state, meaning all structural and logical rules (integrity constraints) are followed.

Deep Dive: The WAL and the "Steal/No-Force" Policy

Most modern DBMS use an architecture described as **Steal / No-Force** to optimize performance:

- **Steal:** The Buffer Manager is allowed to "steal" a page from an active, uncommitted transaction and write it to disk to make room for other data. This requires an **Undo Log** to reverse the change if the transaction later aborts.
- **No-Force:** The DBMS is NOT required to "force" every modified page to disk exactly when a transaction commits. This is much faster as it allows grouped disk writes. This requires a **Redo Log** to re-play the changes if a crash happens while the data was only in RAM.

Together, this is handled by the **Write-Ahead Log (WAL)** protocol: Never write a data page to disk before the corresponding log record is safely stored in non-volatile memory.

Stable Web Resources

- Wikipedia: ACID Properties
- GeeksforGeeks: Transaction States

Quick Revision Cheat Sheet

- **A:** Atomicity (All or None). **C:** Consistency (Valid State). **I:** Isolation (Hidden Work). **D:** Durability (Permanent).
- **Commit:** Permanent. **Rollback:** Abort/Undo. **Savepoint:** Partial Undo.
- **WAL:** Essential for crash recovery (Undo/Redo).
- **Checkpoint:** Shortens recovery time by flushing buffers to disk periodically.
- **Partially Committed:** Execution finished, but not yet hardened to disk.

END OF SUBTOPIC 09 — MOVING TO SUBTOPIC 10

Subtopic 10: Concurrency Control (15 MCQs)

Quick Notes

- **The Goal:** To allow multiple transactions to execute simultaneously while maintaining database consistency and isolation.
- **Concurrency Problems:**
  - *Lost Update:* Two transactions update the same data; the second overwrites the first.
  - *Dirty Read:* A transaction reads data updated by an uncommitted transaction.
  - *Non-Repeatable Read:* A transaction reads the same row twice and gets different values because another transaction committed an update.
  - *Phantom Read:* A transaction executes a query twice and gets a different number of rows because another transaction inserted/deleted rows.
- **Locking Protocols:** Shared (S) locks for reading, Exclusive (X) locks for writing.
  - *Two-Phase Locking (2PL):* Growing Phase (acquiring locks) and Shrinking Phase (releasing locks). Prevents conflict serializability issues but NOT deadlocks.
  - *Strict 2PL:* All Exclusive locks are held until the transaction commits or aborts. Prevents cascading rollbacks.
- **Serializability:** A schedule is serializable if its effect is the same as some serial execution of the same transactions.

Common Traps & Confusions

- **Deadlock vs. Starvation:** A deadlock is a circular wait where no one can move. Starvation is where one transaction is repeatedly passed over and waits indefinitely while others proceed.
- **2PL does NOT mean No Deadlocks:** A common exam trap is assuming 2PL prevents deadlocks. In fact, 2PL can easily lead to deadlocks. Protocols like "Wait-Die" or "Wound-Wait" are needed to prevent them.

- **Isolation Levels:** `READ COMMITTED` is the default in many DBs (like Postgres), which prevents Dirty Reads but still allows Non-Repeatable and Phantom reads.

MCQs

Q1. (Scenario-Based) User A and User B are both trying to buy the last ticket for a concert. 1. User A reads the ticket count (Count = 1). 2. User B reads the ticket count (Count = 1). 3. User A decrements the count and writes (Count = 0). 4. User B decrements the count and writes (Count = 0). Both users receive a "Success" message, but only one physical ticket exists. Which concurrency problem occurred?

A) Dirty Read

B) Phantom Read

C) Lost Update

D) Non-Repeatable Read

> This is a classic Lost Update. User B's update was based on a stale version of the data (Count=1), and when User B wrote the result, it effectively erased the fact that User A had already "taken" the ticket.

Q2. (Code-Tracing) Analyze the following SQL execution sequence between two sessions. What will Transaction 1 see at Time T3?

sql-isolation-test

```sql
-- Transaction 1 (Isolation Level: READ COMMITTED)
T1: SELECT balance FROM accounts WHERE id = 1; -- Result: 500

-- Transaction 2
T2: UPDATE accounts SET balance = 1000 WHERE id = 1;
T2: COMMIT;

-- Transaction 1 again
T3: SELECT balance FROM accounts WHERE id = 1;
```

A) 500

B) 1000

C) NULL

D) Error: Transaction must be restarted.

> In `READ COMMITTED` isolation, a transaction sees any data committed by others as soon as it happens. Between T1 and T3, Transaction 2 committed an update, so T1 sees the new value (1000). This is a "Non-Repeatable Read," which is permitted in this isolation level.

Q3. (Conceptual) A schedule is considered **Conflict Serializable** if:

A) It contains no write operations.

B) All transactions are executed one after another in any order.

C) Its precedence graph (where nodes are transactions and edges represent conflicting operations) is acyclic.

D) It uses a Two-Phase Locking protocol.

> Conflict serializability is determined by the Precedence Graph. If Transaction T1 accesses data X and then T2 accesses data X (where at least one is a write), an edge T1 → T2 is drawn. If the graph has no cycles, the schedule is conflict serializable.

Q4. (Scenario-Based) A transaction follows the **Basic Two-Phase Locking (2PL)** protocol. It has just released a shared lock on Table A. Can it now acquire an exclusive lock on Table B?

A) Yes, if it is still in the Growing Phase.

B) No, because it has entered the Shrinking Phase.

C) Yes, exclusive locks are always allowed.

D) No, 2PL only allows one lock at a time.

> Under 2PL, a transaction cannot acquire any new locks once it has released its first lock. Releasing a lock marks the end of the "Growing Phase" and the start of the "Shrinking Phase."

Q5. (Calculation) In a database using the **Wait-Die** scheme for deadlock prevention, Transaction T1 (Timestamp 10) requests a lock held by Transaction T2 (Timestamp 20). What happens?

A) T1 waits for T2.

B) T1 dies (aborts).

C) T2 dies (aborts).

D) Both transactions wait.

> In Wait-Die: If an older transaction (lower timestamp, e.g., 10) requests a resource held by a younger transaction (higher timestamp, e.g., 20), the older one is allowed to wait. If a younger one requests from an older one, the younger one "dies" (aborts).

Q6. (Code-Tracing) Which of the following SQL statements explicitly requests an **Exclusive Lock** on the selected rows, preventing other transactions from even reading them in most locking-based engines?

A) SELECT * FROM products WHERE id = 10;

B) SELECT * FROM products WHERE id = 10 LOCK IN SHARE MODE;

C) SELECT * FROM products WHERE id = 10 FOR UPDATE;

D) UPDATE products SET stock = stock - 1 WHERE id = 10;

> `SELECT ... FOR UPDATE` is used for "Pessimistic Locking." It places an Exclusive (X) lock on the rows. While a normal UPDATE also locks the row, `FOR UPDATE` is specifically used to lock the row during a SELECT so that the application can calculate a change before writing it back.

Q7. (Conceptual) What is the primary benefit of **Strict Two-Phase Locking (Strict 2PL)** over Basic 2PL?

A) It eliminates the possibility of deadlocks.

B) It requires fewer locks overall.

C) It ensures that schedules are cascadeless (prevents cascading rollbacks).

D) It allows for higher concurrency by releasing locks early.

> Strict 2PL holds all exclusive locks until the transaction commits or aborts. This ensures that no other transaction can read "dirty" uncommitted data. If a transaction fails, it can be rolled back without needing to roll back any other transactions that might have read its data.

Q8. (Scenario-Based) An application uses **Optimistic Concurrency Control (OCC)**. A transaction spends a long time performing calculations in its local buffer. When it finally tries to commit, it is told that the underlying data has changed, and it must restart. At which phase did this failure occur?

A) Read Phase

B) Validation Phase

C) Write Phase

D) Recovery Phase

> OCC has three phases: 1. Read (work on a local copy), 2. Validation (check if any other transaction modified the data in the meantime), and 3. Write (apply changes to the DB). The check happens during the Validation Phase.

Q9. (Calculation) If there are 3 transactions (T1, T2, T3) executing concurrently, how many possible **Serial Schedules** exist?

A) 3

B) 6

C) 9

D) 27

> A serial schedule executes transactions one after another in some order. For n transactions, there are $n!$ possible serial schedules. $3! = 3 \times 2 \times 1 = 6$.

Q10. (Conceptual) The **Thomas Write Rule** is an optimization of the Basic Timestamp Ordering protocol. What does it do when a transaction Ti tries to write to a data item that already has a Write-Timestamp newer than Ti's own timestamp?

A) Ti is aborted and restarted.

B) Ti is forced to wait until the newer transaction commits.

C) Ti's write is simply ignored, and the transaction continues.

D) The database system crashes to prevent inconsistency.

> The Thomas Write Rule handles out-of-order writes. If a newer transaction has already written a value, an older transaction's write is "obsolete." Instead of aborting the older transaction, we simply ignore its write, as the newer value is already the "current" one.

Q11. (Comparison) In the context of database locking, what is the difference between **Deadlock** and **Starvation**?

A) Deadlock is for SQL; Starvation is for NoSQL.

B) Deadlock can be fixed by waiting; Starvation requires a reboot.

C) Deadlock is a circular wait where no transaction can proceed; Starvation is when a transaction waits indefinitely while others are constantly granted access.

D) They are different terms for the same problem.

> Deadlock is a "freeze" involving at least two transactions. Starvation is a scheduling "unfairness" where a specific transaction never gets the resource it needs because other transactions keep cutting in line (often due to priority or unlucky timestamping).

Q12. (Scenario-Based) A report runs `SELECT SUM(salary) FROM Employees`. Halfway through the scan, a concurrent transaction updates an employee's salary that the report has already scanned, and another employee's salary that the report has *not yet* scanned. The final sum is inconsistent. This is an example of:

A) Dirty Read
B) Phantom Read
C) Inconsistent Analysis (or Fuzzy Read)
D) Lost Update

> Inconsistent Analysis occurs when a transaction (usually an aggregation) reads a partial state of the database while other updates are occurring, leading to a result that never existed in any single consistent version of the database.

Q13. (Calculation) Which of the following lock requests would be **GRANTED** if the data item is currently held under a **Shared (S) Lock** by another transaction?

A) A request for another Shared (S) Lock.
B) A request for an Exclusive (X) Lock.
C) A request to Upgrade the existing S-lock to an X-lock.
D) No locks are granted if one is already held.

> Shared locks are compatible with other Shared locks. Multiple transactions can read the same data simultaneously. Exclusive locks, however, are incompatible with any other locks.

Q14. (Code-Tracing) A system is using **Multi-Version Concurrency Control (MVCC)**. When Transaction T1 (starts at Time 10) reads a row that was updated by Transaction T2 (commits at Time 15), what version of the row does T1 see?

A) The version of the row as it existed at Time 10.
B) The updated version committed at Time 15.
C) No version (the read is blocked).
D) A NULL value.

> MVCC provides a "snapshot" of the database. A transaction sees a version of the data consistent with its start time. This allows reads to happen without ever blocking writes, which is the primary advantage of MVCC (used in Postgres, MySQL InnoDB, etc.).

Q15. (Comparison) When comparing **Pessimistic** and **Optimistic** locking, which is more suitable for a system with **high data contention** (many users trying to update the same rows)?

A) Pessimistic Locking, because it prevents conflicts by locking data before work begins.

B) Optimistic Locking, because it avoids the overhead of managing locks.

C) Both are equally suitable.

D) Neither; high contention requires a NoSQL approach.

> In high contention, Optimistic locking would result in too many transaction restarts (aborts during validation), wasting CPU. Pessimistic locking (using actual locks) is better because it forces transactions to wait in a queue rather than constantly failing and retrying.

Deep Dive: The Precedence Graph and Serializability

To determine if a concurrent schedule is **Conflict Serializable**, we build a Directed Graph (Precedence Graph):

- For every transaction T_i and T_j, an edge $T_i \rightarrow T_j$ exists if:
  1. T_i executes `Read(X)` and T_j later executes `Write(X)`.
  2. T_i executes `Write(X)` and T_j later executes `Read(X)`.
  3. T_i executes `Write(X)` and T_j later executes `Write(X)`.
- If this graph contains **any cycle** (e.g., $T1 \rightarrow T2 \rightarrow T1$), the schedule is **not** conflict serializable.
- **View Serializability:** A schedule might be View Serializable even if it's not Conflict Serializable. View serializability is harder to check (it's an NP-Complete problem) and focuses on ensuring each transaction reads the same values it would in a serial schedule. All conflict-serializable schedules are also view-serializable.

Stable Web Resources

- Wikipedia: Concurrency Control
- PostgreSQL Documentation: MVCC

Quick Revision Cheat Sheet

- **Lost Update:** Overwriting someone's work. **Dirty Read:** Reading uncommitted work.
- **S-Lock:** Shared (Read). **X-Lock:** Exclusive (Write).
- **2PL:** Growing (get locks) → Shrinking (release locks).
- **Strict 2PL:** Hold X-locks until the very end (Prevents cascading aborts).
- **Wait-for Graph:** Cycles here mean a **Deadlock** exists.
- **Isolation Levels:** Serializable (Strongest) → Repeatable Read → Read Committed → Read Uncommitted.
- **MVCC:** Reads don't block writes; writes don't block reads.

END OF SUBTOPIC 10 — MOVING TO SUBTOPIC 11

Subtopic 11: Recovery Management (10 MCQs)

Quick Notes

- **Recovery Goal:** To restore the database to the most recent consistent state that existed before a failure occurred.
- **Log-Based Recovery:** Records all database modifications in a stable log.
  - *Deferred Update:* Database is not updated until the transaction commits. Only **Redo** is needed.
  - *Immediate Update:* Database is updated as the transaction executes. Both **Undo and Redo** are needed.

- **Write-Ahead Logging (WAL):** The fundamental rule that log records representing a change must be flushed to stable storage *before* the actual data page is written to disk.
- **Checkpoint:** A point in the log where the DBMS ensures all buffers are flushed to disk. It drastically shortens the recovery time by limiting how far back the log must be scanned.
- **Idempotency:** Recovery operations (Redo/Undo) must be idempotent; executing them multiple times must result in the same state as executing them once.

Common Traps & Confusions

- **Undo vs. Redo Logic:** If a transaction has a [Start] but no [Commit] or [Abort] in the log at the time of crash, it must be **Undone**. If it has both [Start] and [Commit], it must be **Redone**.
- **Checkpoint Fallacy:** Many students think a checkpoint stops all transactions. Modern databases use "Fuzzy Checkpoints" which allow transactions to continue running while dirty pages are being flushed to disk.
- **Storage Failure:** Recovery management usually refers to "System Failures" (crash/power loss). If the physical disk fails ("Media Failure"), the only recovery path is restoring from a **Backup** and then replaying the logs.

MCQs

Q1. (Scenario-Based) A database server crashes at Time T. The recovery manager inspects the log and finds the following sequence. Which action should the manager take for Transactions T1 and T2?

db-recovery-log

```
[08:00] <T1, Start>
[08:05] <T1, Write, A, 100, 200>
[08:10] <T2, Start>
[08:15] <T1, Commit>
[08:20] <T2, Write, B, 50, 75>
-- SYSTEM CRASH --
```

A) Redo T1 and Redo T2

B) Undo T1 and Redo T2

C) Redo T1 and Undo T2

D) Undo T1 and Undo T2

> T1 has a [Commit] entry, meaning its changes were logically completed and must be re-applied to the data files (Redo). T2 was active but did not commit before the crash; its partial changes must be reversed (Undo) to maintain atomicity.

Q2. (Code-Tracing) In a database using the **Deferred Update** (No-Undo/Redo) protocol, what will be the state of the physical data on disk if a transaction modifies an item but the system crashes *before* the `COMMIT` command is executed?

A) The item will have the new value, and the recovery manager must undo it.

B) The item will still have the old value because no writes were "forced" to the data files yet.

C) The item will be corrupted.

D) The database will automatically delete the table.

> Deferred update ensures that no changes are written to the actual database files until the transaction reaches its commit point. If a crash occurs before commit, the data on disk is still in its original state, so no "Undo" is necessary. Recovery only involves re-applying (Redoing) transactions that successfully committed but whose changes were only in the RAM buffer.

Q3. (Conceptual) The **Write-Ahead Logging (WAL)** protocol dictates that:

A) Data must be written to the database before the log is updated.

B) Logs and data must be written to the same physical disk sector simultaneously.

C) The log record for a modification must be written to stable storage before the modified data page is written to the database.

D) Transactions cannot commit until all data is written to the primary database files.

> WAL is the golden rule of recovery. If the database page were written first and the system crashed before the log was written, the recovery manager would have no way to "Undo" that change, violating atomicity.

Q4. (Calculation) A database performs a **Checkpoint** every 30 minutes. If the system crashes 29 minutes after the last checkpoint, which part of the log must the recovery manager scan?

A) The entire log from the beginning of time.

B) Only the last 1 minute of the log.

C) From the last checkpoint to the end of the log (plus any active transactions at the time of the checkpoint).

D) No scanning is required; the database is already consistent.

> A checkpoint guarantees that all transactions that committed *before* the checkpoint have their changes safely on disk. The recovery manager only needs to process log records from the checkpoint forward, though it must also look back to find the [Start] records of transactions that were still active during the checkpoint.

Q5. (Scenario-Based) A data center experiences a catastrophic fire that physically destroys the primary database hard drives. To recover the data, which of the following is strictly required?

A) The Write-Ahead Log (WAL) stored on the same drive.

B) The Buffer Pool in the system RAM.

C) An archived full backup and all transaction logs stored on a separate, surviving medium.

D) A fresh installation of the DBMS software.

> Recovery from "Media Failure" is different from "System Failure." Since the physical disks are gone, the internal logs on those disks are also gone. Recovery requires restoring the most recent off-site backup and then "Rolling Forward" by replaying all transaction logs generated since that backup.

Q6. (Conceptual) Why must the **Redo** and **Undo** operations in a recovery algorithm be **Idempotent**?

A) To ensure that they only use O(1) space.

B) To allow the database to encrypt the logs.

C) Because the recovery process itself might crash and be restarted multiple times.

D) To prevent users from accessing the data during recovery.

> If the system crashes *during* recovery, the recovery process will start again from the last checkpoint. This means some "Redo" operations might be performed a second time. If they weren't idempotent (e.g., "Add 5 to Balance" is not idempotent, whereas "Set Balance to 500" is), the data would become incorrect.

Q7. (Comparison) What is the primary difference between a **System Crash** and an **Aborted Transaction** from a recovery perspective?

A) Aborted transactions require Redo; crashes require Undo.

B) Aborted transactions only require Undo for that specific transaction; a crash requires analysis and recovery for all active transactions.

C) Crashes are managed by the user; Aborts are managed by the OS.

D) There is no difference in how the DBMS handles them.

> When a single transaction aborts (e.g., due to a constraint violation), the DBMS simply uses the log to "Undo" that specific transaction. When a system crash occurs, the entire state of the RAM buffer is lost, requiring a full scan of the log to determine which transactions need Redo and which need Undo.

Q8. (Code-Tracing) Consider the following log entry for an **Immediate Update** protocol: `<T1, X, 50, 100>`. During the **Undo** phase of recovery, what value is written to data item X?

A) 100

B) 50

C) 150

D) NULL

> In an Immediate Update log record `<T, Item, Old_Val, New_Val>`, the Undo operation uses the `Old_Val` (50) to restore the state, while the Redo operation uses the `New_Val` (100).

Q9. (Calculation) In the **ARIES** recovery algorithm, the "Analysis Phase" creates two tables. What are they?

A) The Primary Key Table and the Foreign Key Table.

B) The Transaction Table (active transactions) and the Dirty Page Table (pages in RAM not yet on disk).

C) The Commit Table and the Rollback Table.

D) The User Table and the System Table.

> The Analysis Phase of ARIES scans the log from the last checkpoint to the end to identify: 1. Which transactions were active at the time of the crash (Transaction Table) and 2. Which data pages in the buffer were "dirty" and might not have been written to disk (Dirty Page Table).

Q10. (Conceptual) What is the purpose of the **Log Sequence Number (LSN)** in modern recovery management?

A) To uniquely identify each log record and allow the DBMS to compare the "age" of a data page with the "age" of the log.

B) To count the number of users logged into the system.

C) To sort the results of a `SELECT` query.

D) To encrypt the communication between the client and server.

> Each data page in the database contains the LSN of the last log record that modified it. During recovery, the DBMS checks the page's LSN; if the page's LSN is greater than or equal to the LSN of the Redo log record, the DBMS knows the change is already on disk and can skip that Redo operation.

Deep Dive: The ARIES Recovery Algorithm

Most industrial-strength databases (DB2, SQL Server) use the **ARIES (Algorithms for Recovery and Isolation Exploiting Semantics)** method. It operates in three distinct phases:

1. **Analysis Phase:** Starts from the last Checkpoint. It determines which pages were dirty (not on disk) and which transactions were active at the time of the crash.
2. **Redo Phase:** It repeats history. It starts from the oldest dirty page and re-applies all changes in the log (even those for transactions that eventually aborted). This brings the database to the exact state it was in at the moment of the crash.
3. **Undo Phase:** It scans backward from the end of the log and reverses the changes of all transactions that were active (uncommitted) at the time of the crash.

The genius of ARIES is **Logging during Undo**: When it reverses a change, it writes a "Compensation Log Record" (CLR). If the system crashes again *during* the recovery, the CLR ensures the system doesn't try to undo the same thing twice, preventing infinite loops in recovery.

Stable Web Resources

- [Wikipedia: Write-Ahead Logging]

- GeeksforGeeks: Log-Based Recovery

Quick Revision Cheat Sheet

- **WAL:** Log before Data.
- **Commit:** Log must be on disk before 'Success' is sent to user.
- **Undo:** Reverts active transactions (uses Old_Value).
- **Redo:** Re-applies committed transactions (uses New_Value).
- **Checkpoint:** Flushes buffers to disk; reduces log scan time.
- **Idempotency:** Redo(Redo(x)) = Redo(x).

END OF SUBTOPIC 11 — MOVING TO SUBTOPIC 12

Subtopic 12: Indexing & File Organization (15 MCQs)

Quick Notes

- **The Purpose:** Indexing is a data structure technique used to minimize the number of disk I/O operations required to locate specific data.
- **B-Tree vs. B+ Tree:** In a B-Tree, data and pointers exist in all nodes. In a **B+ Tree** (the standard for RDBMS), actual data records (or pointers to them) exist **only in leaf nodes**. Leaf nodes are linked together for fast sequential range scans.
- **Clustered Index:** The actual physical rows in the table are sorted based on the index key. A table can have **only one** clustered index (usually the Primary Key).
- **Non-Clustered (Secondary) Index:** A separate structure containing the key and a pointer (RID) to the actual data row. A table can have many non-clustered indexes.
- **Hashing:** Best for exact matches (O(1) average), but extremely poor for range queries (e.g., `WHERE price > 100`).
- **File Organizations:** Heap (unordered), Sequential (sorted by key), Hashed (spread by hash function), and Clustered.

Common Traps & Confusions

- **Index on Every Column:** While indexes speed up `SELECT`, they significantly **slow down** `INSERT`, `UPDATE`, and `DELETE` because the index structure must be updated synchronously.
- **Primary Index vs. Primary Key:** Usually, they are the same, but they are different concepts. The Key is a logical constraint; the Index is a physical storage optimization.
- **Index Selectivity:** An index on a "Gender" column (Low Selectivity) is often useless because the engine might decide a full table scan is faster than jumping back and forth between the index and the data.
- **B+ Tree Fan-out:** High fan-out (many children per node) is the goal, as it keeps the tree short (usually 3-4 levels for millions of rows), resulting in fewer disk reads.

MCQs

Q1. (Scenario-Based) An e-commerce platform's "Product Search" is slow when users filter by "Category" and "Price". The DBA creates a single index on `(category, price)`. Now, searches for `WHERE category = 'Toys'` are fast, but searches for `WHERE price = 50.00` are still slow. Why?

A) The index is corrupted and needs rebuilding.

B) The "Leftmost Prefix" rule: Composite indexes can only be used if the first column of the index is present in the query filter.

C) Price is a floating-point number, which cannot be indexed in a B+ Tree.

D) Category has too many duplicates, making the index ineffective.

> In a composite index on (A, B), the data is sorted by A first, then by B. Searching by B alone is like searching for a name in a phonebook using only the first name—the alphabetical sorting of the last names (the first column) makes the search impossible without a full scan.

Q2. (Comparison) Why do modern RDBMS (like MySQL and SQL Server) utilize **B+ Trees** instead of standard B-Trees for disk-based indexing?

A) B+ Trees use less RAM than B-Trees.

B) B-Trees do not support duplicate keys.

C) B+ Trees store all data in leaf nodes and link them, allowing for highly efficient

range-based scans and higher fan-out.

D) B-Trees are strictly for memory-based storage, while B+ Trees are for SSDs.

> Because B+ Trees store data only at the leaf level, internal nodes can fit more keys per block (higher fan-out), making the tree shallower. Furthermore, the linked list connecting leaf nodes allows the engine to perform range queries (e.g., `BETWEEN`) by finding the first leaf and then just following the pointers.

Q3. (Code-Tracing) Analyze the following execution plan output. Why did the database engine decide to ignore the index on the `status` column?

mysql - explain_output

```
EXPLAIN SELECT * FROM orders WHERE status != 'SHIPPED';

-- Output:
type: ALL (Full Table Scan)
Possible Keys: idx_status
key: NULL
```

A) Because the `status` column contains NULL values.

B) Because `SHIPPED` is a reserved keyword in SQL.

C) Because inequality operators (!=, <>) usually prevent the engine from utilizing B-tree indexes efficiently.

D) Because the table has fewer than 10 rows.

> B-tree indexes are optimized for finding specific values or ranges. Most optimizers will skip an index for "NOT EQUAL" (`!=`) queries because "everything except X" usually involves a large portion of the table, making a sequential scan more efficient than random-access index jumps.

Q4. (Calculation) A database block is 4KB (4096 bytes). An index entry consists of a 4-byte integer key and an 8-byte child pointer. Ignoring header overhead, what is the maximum **fan-out** (number of children) of an internal node in this B+ Tree?

A) 1024

B) 512

C) 341

D) 256

> Each internal node has n pointers and $n-1$ keys.
> Formula: $(n \times 8) + ((n-1) \times 4) \le 4096$.
> $8n + 4n - 4 \le 4096 \rightarrow 12n \le 4100 \rightarrow n \le 341.6$.
> The fan-out is 341.

Q5. (Conceptual) Which of the following is the defining characteristic of a **Clustered Index**?

A) It is the index that stores the most frequently accessed columns.

B) It is a bitmapped index used for Boolean data types.

C) It determines the physical order of data rows on the disk.

D) It is an index that can be created on multiple tables simultaneously.

> In a clustered index, the "leaf" of the index *is* the actual data row. Since physical rows on a disk can only be sorted in one way, you can have only one clustered index per table.

Q6. (Scenario-Based) A database contains millions of "Sensor Readings". Queries always look for readings within a specific 10-minute time window (e.g., `WHERE reading_time BETWEEN '10:00' AND '10:10'`). Which index type is the most mathematically efficient for this use case?

A) Hash Index

B) B+ Tree Index

C) Linear Probing

D) Inverted Index

> Range queries (`BETWEEN`, >, <) require an ordered data structure. B+ Trees maintain keys in sorted order, allowing the engine to find the start of the range and scan sequentially. Hash indexes are unsuitable because they randomize the location of keys.

Q7. (Comparison) What is the main difference between a **Dense Index** and a **Sparse Index**?

A) A Dense Index has an entry for every search key in the data file; a Sparse Index only has entries for some search keys (e.g., one per block).

B) Dense indexes are for strings; Sparse indexes are for numbers.

C) Sparse indexes are faster for insertion; Dense indexes are faster for deletion.

D) Dense indexes only work on clustered tables.

> In a Dense Index, every record is represented in the index. In a Sparse Index, we only store a pointer to the start of each block. Sparse indexes use less space but require the data file to be sorted by the search key.

Q8. (Code-Tracing) You execute the following command to optimize a report. What is the technical term for the type of index created?

sql-shell - optimization

```sql
CREATE INDEX idx_report
ON sales(sale_date, region)
INCLUDE (total_amount);
```

A) Clustered Index

B) Bitmap Index

C) Covering Index (or Index with Included Columns)

D) Reverse Key Index

> By "including" `total_amount`, the index now contains all the columns needed for a query like `SELECT total_amount FROM sales WHERE sale_date = '...'`. The engine can return the result directly from the index without ever reading the actual table (Heap), which is called an "Index-Only Scan."

Q9. (Calculation) A table is stored as an unordered **Heap File** with 1,000,000 blocks. If no index exists, what is the average number of block reads required to find a record by its Primary Key?

A) 1

B) 1,000

C) 500,000

D) 1,000,000

> In a heap file (unordered), the only way to find a record is a linear search. On average, the record will be found after scanning half the blocks (N/2). In the worst case, 1,000,000 reads.

Q10. (Conceptual) Why is it often recommended to use a **Surrogate Key** (like an Auto-incrementing Integer) as a Clustered Index rather than a natural key (like a UUID or Name)?

A) Integers take less disk space than strings.

B) Sequential values prevent "Page Splitting" in the B+ Tree, keeping the tree balanced and reducing fragmentation.

C) Integers are the only data type supported by B+ Trees.

D) Surrogate keys automatically encrypt the data.

> When you insert random values (like a UUID) into a clustered index, the engine must insert rows in the middle of pages. If a page is full, it must "split" the page to make room, which is an expensive operation and leads to low "Fill Factor" and fragmentation. Sequential integers always append to the end.

Q11. (Scenario-Based) A "User Analytics" table has a `membership_type` column with only 3 possible values: 'Free', 'Silver', 'Gold'. Most queries aggregate data based on this column. Which index type is specifically designed for such "Low-Cardinality" data?

A) B+ Tree Index

B) Bitmap Index

C) Hash Index

D) Spatial Index

> Bitmap indexes use bit arrays (1s and 0s) to represent the presence of a value. They are extremely space-efficient for columns with very few unique values (low cardinality) and allow for lightning-fast logical operations (AND, OR, NOT) when multiple filters are applied.

Q12. (Calculation) If a B+ Tree index has a height of 3 (Root, 1 Intermediate level, 1 Leaf level) and the table is NOT clustered, how many I/O operations are required to retrieve a single row using this index?

A) 3

B) 4

C) 2

D) 1

> The engine must read 3 index blocks (Root → Intermediate → Leaf). Since it's a non-clustered index, the leaf only contains a pointer to the data. One final I/O is required to fetch the actual row from the data file. Total = 4.

Q13. (Comparison) What is the primary disadvantage of **Static Hashing** in a growing database?

A) It uses too much CPU for the hash function.

B) It is slower than B+ Trees for exact lookups.

C) As the data grows, the number of collisions increases, leading to long overflow chains and performance degradation.

D) It cannot be used with integer keys.

> Static hashing has a fixed number of buckets. As data exceeds the initial capacity, many keys map to the same bucket, forcing the use of "overflow chains" (linked lists), which makes search time linear (O(N)) instead of constant (O(1)). Dynamic/Extendible hashing solves this by splitting buckets.

Q14. (Conceptual) In indexing theory, what does **"Index Selective"** refer to?

A) The ability of the index to handle NULL values.

B) The number of columns included in a composite index.

C) The ratio of the number of distinct values in the index to the total number of records in the table.

D) The speed at which an index can be rebuilt after a crash.

> Selectivity = (Distinct Values / Total Rows). An index on a Primary Key has a selectivity of 1.0 (Highest). High selectivity means the index is very effective at pinpointing specific rows. Low selectivity (like Gender) often leads the Query Optimizer to ignore the index.

Q15. (Debugging/Analysis) A developer notices that their index on `hire_date` is not being used in the following query. What is the cause?

sql-shell - performance_bug

```sql
SELECT * FROM employees
WHERE YEAR(hire_date) = 2023;
```

A) The YEAR function is not supported in the WHERE clause.

B) hire_date is a TIMESTAMP, not a DATE.

C) Using a function on an indexed column ("Non-SARGable" query) prevents the engine from using the index.

D) The index was created using ASC but the query needs DESC.

> When you wrap a column in a function (like `YEAR()`), the engine cannot use the B-tree to find the value because the index is built on the raw dates, not the transformed year values. This is called a Non-SARGable (Search ARGumentable) query. Better: `WHERE hire_date >= '2023-01-01' AND hire_date <= '2023-12-31'`.

Deep Dive: B+ Tree Maintenance and Page Splitting

When you insert a row into a table with a B+ Tree index, the engine traverses the tree to find the correct leaf page. If that page is full (usually kept at 80% to allow for some growth), a **Page Split** occurs:

1. The engine allocates a new empty page.
2. It moves approximately half of the entries from the original page to the new one.
3. It inserts a new entry into the parent node to point to the new page.
4. If the parent is also full, the split propagates upward (potentially all the way to the root, increasing the height of the tree).

This is why **Clustered Index fragmentation** happens. Frequent splits lead to pages that are physically scattered across the disk, making sequential scans much slower. Regularly "Rebuilding" or "Reorganizing" indexes defragments the pages and restores sequential order.

Stable Web Resources

- Wikipedia: B+ Tree
- PostgreSQL: Index Types

Quick Revision Cheat Sheet

- **B+ Tree:** Standard DB index. Data in leaves, leaves linked. Best for range queries.
- **Hash Index:** Fast O(1) for = queries. Useless for >, <, or `BETWEEN`.
- **Clustered:** Data sorted by key. Max 1 per table.
- **Non-Clustered:** Pointer to data. Multiple per table allowed.
- **SARGable:** Keep indexed columns "naked" in queries (no functions) to ensure index usage.

- **Low Cardinality:** Use Bitmap indexes (if supported).
- **Composite Index:** Order of columns matters (Leftmost prefix rule).

END OF SUBTOPIC 12 — MOVING TO SUBTOPIC 13

Subtopic 13: Query Processing & Optimization (10 MCQs)

Quick Notes

- **The Query Lifecycle:**
  1. *Parsing & Translation:* Checks syntax and translates SQL into a Relational Algebra expression (Query Tree).
  2. *Optimization:* Evaluates multiple execution plans to find the "cheapest" one.
  3. *Execution:* The engine runs the chosen plan and returns results.
- **Cost-Based Optimizer (CBO):** Uses database statistics (row counts, index selectivity, data distribution) to estimate the cost (I/O and CPU) of different plans.
- **Rule-Based/Heuristic Optimization:** Uses a fixed set of rules, such as "Perform Selection (Filter) as early as possible" to reduce the size of intermediate results.
- **Join Ordering:** For multiple joins, the optimizer decides which tables to join first to minimize the size of temporary tables in RAM.
- **Materialization vs. Pipelining:** Materialization writes intermediate results to disk; Pipelining passes results directly from one operation to the next in memory.

Common Traps & Confusions

- **Declarative vs. Procedural:** SQL is declarative (you say *what*). The Query Optimizer makes it procedural (it decides *how*).
- **Stale Statistics:** If a table grows from 100 rows to 1,000,000 rows but the "Statistics" aren't updated, the optimizer might still think a Full Table Scan is faster than an Index Scan, causing massive performance lag.

- **The "Magic" of the Optimizer:** The optimizer is not perfect. Sometimes a "hint" (like `FORCE INDEX`) is required to override its decision if the statistics are misleading.

MCQs

Q1. (Conceptual) During the **Query Translation** phase, into which mathematical representation is a declarative SQL statement typically converted?

A) A binary machine code file.

B) A JSON-based configuration object.

C) An internal Relational Algebra expression (Query Tree).

D) A physical block mapping on the hard drive.

> SQL is a high-level language. To process it, the DBMS translates it into Relational Algebra, which provides a sequence of operations (Select, Project, Join) that the engine can actually execute step-by-step.

Q2. (Scenario-Based) A heuristic-based query optimizer encounters a query joining three large tables. Which rule is it most likely to apply first to reduce the overall execution time?

A) Perform all Joins first, then apply filters.

B) Perform Selection (WHERE) and Projection (SELECT) operations as early as possible.

C) Sort the tables by their Primary Keys before joining.

D) Convert all Inner Joins into Full Outer Joins for safety.

> One of the most powerful heuristic rules is "Selection Pushing." By filtering rows (Selection) and removing unnecessary columns (Projection) early, the engine reduces the size of the data it has to process in the expensive Join phases.

Q3. (Code-Tracing) Analyze the output of the `EXPLAIN` command below. What does the **"cost"** value (e.g., `0.00..10.50`) represent to the optimizer?

postgres - plan_analyzer

```
EXPLAIN SELECT * FROM users WHERE id = 1;

-- Output:
```

```
Index Scan using pk_users on users (cost=0.00..10.50 rows=1 width=40)
```

A) The total time in milliseconds taken to run the query.

B) The number of bytes of data transferred over the network.

C) A dimensionless unit estimating the I/O and CPU resources required to execute the plan.

D) The security clearance level required to view the data.

> The "cost" is an estimate, not a real time. It usually represents the number of disk page fetches and CPU cycles. The optimizer compares the costs of multiple plans and chooses the one with the lowest total cost.

Q4. (Conceptual) In a **Cost-Based Optimizer (CBO)**, where does the engine retrieve the information needed to estimate the cost of a plan (e.g., number of rows, distinct values)?

A) It performs a full count of every table before every query.

B) It consults the System Catalog, which stores metadata and statistics about the tables and indexes.

C) It asks the operating system's file manager.

D) It uses random numbers to simulate data distribution.

> A CBO relies on the Data Dictionary/System Catalog. This catalog stores statistics (histogram of values, row counts, height of B+ trees) that are updated periodically (e.g., via the `ANALYZE` or `RUNSTATS` commands).

Q5. (Scenario-Based) A query that was previously fast suddenly becomes extremely slow after 1 million new rows are inserted into the table, even though an index exists. What is the most likely cause?

A) The index was automatically deleted by the DBMS.

B) The database engine switched from 64-bit to 32-bit.

C) The database statistics are "stale" (out of date), causing the optimizer to choose a poor execution plan.

D) SQL queries lose efficiency as the table ID increases.

> If statistics aren't updated, the optimizer might still think the table is small and choose a "Full Table Scan." Updating the statistics allows the optimizer to "see" the new data and realize that using the index is now significantly faster.

Q6. (Comparison) What is the difference between **Materialization** and **Pipelining** in the context of query execution?

A) Materialization is for SELECT; Pipelining is for UPDATE.

B) Materialization stores intermediate results on disk; Pipelining passes results from one operator to another without temporary storage.

C) Materialization is faster than Pipelining.

D) Pipelining is only supported in NoSQL databases.

> Pipelining is generally faster as it avoids slow disk I/O. As rows are produced by one operation (e.g., a Scan), they are immediately "piped" into the next operation (e.g., a Filter) in memory. Materialization is only used when the intermediate result is too large for RAM or must be sorted.

Q7. (Calculation/Logic) The optimizer is evaluating an **INNER JOIN** between Table A (1,000 rows) and Table B (1,000,000 rows). Which join order is it most likely to attempt to minimize memory usage for a Hash Join?

A) Use Table A as the "Build" table (the hash table in RAM) and scan Table B.

B) Use Table B as the "Build" table and scan Table A.

C) Join them both into a third temporary table first.

D) Join order does not affect performance in a Hash Join.

> In a Hash Join, the optimizer builds a hash table in RAM for one table. To save memory, it should choose the smaller table (Table A) to be the "Build" table and then stream the larger table (Table B) against it.

Q8. (Conceptual) Why is the **Join Order** one of the most complex parts of query optimization?

A) Because SQL only supports one join per query.

B) Because joins can only be performed on integer columns.

C) Because for N tables, there are an exponential number of possible join sequences (N!).

D) Because the hardware must be reconfigured for each join type.

> Joining multiple tables is associative and commutative. For 4 tables, there are 24 possible orders. For 10 tables, there are over 3.6 million. The optimizer uses "Dynamic Programming" or "Genetic Algorithms" to find a good order without checking every single one.

Q9. (Code-Tracing) A developer notices that a query using `SELECT *` is slower than `SELECT id, name`. From a query processing perspective, why is this?

A) `SELECT *` disables the Query Optimizer.

B) `SELECT *` increases the I/O cost (more data pages must be read) and prevents the engine from using a "Covering Index."

C) `SELECT *` forces the database to re-index the table.

D) SQL engines are programmed to penalize the use of wildcards.

> If the query only needs `id` and `name`, and an index exists on those two columns, the optimizer can perform an "Index-Only Scan" (Pipelining). If `*` is used, the engine must perform a "Bookmark Lookup" to fetch the full row from the disk, adding significant I/O cost.

Q10. (Scenario-Based) A query plan shows a **"Sequential Scan"** on a table of 10 million rows, even though there is a `WHERE id = 500` filter and an index on `id` exists. What could be a technical reason for this?

A) The `id` value 500 is too large for an index.

B) The index was created using `DESC` but the query is `ASC`.

C) The data type in the WHERE clause (e.g., '500' as a string) does not match the column data type (INT), causing an implicit function call that invalidates the index.

D) The Sequential Scan is always faster for Primary Keys.

> This is a "Type Mismatch" error. If the column is an Integer but you provide a String (or vice versa), the DBMS may perform an implicit conversion on every row (like `CAST(id AS VARCHAR) = '500'`). This transformation makes the query Non-SARGable, forcing a full scan.

Deep Dive: Cost-Based Optimization (CBO) Metrics

The "Cost" of a plan is calculated using a cost model that assigns weights to different hardware operations. In a typical model (like PostgreSQL):

- **seq_page_cost (1.0):** The cost to read one 8KB page sequentially from disk.
- **random_page_cost (4.0):** The cost to read one page via a random access (much higher because of disk seek time).
- **cpu_tuple_cost (0.01):** The cost to process one row in the CPU.
- **cpu_index_tuple_cost (0.005):** The cost to process one index entry.

The optimizer calculates: `Total Cost = (Number of Pages * Page Cost) + (Number of Rows * CPU Cost)`. This is why on very small tables (e.g., 5 rows), the optimizer often chooses a Sequential Scan over an Index Scan—the "random_page_cost" of jumping to the index and then to the data is higher than simply reading the one sequential page containing all 5 rows.

Stable Web Resources

- Wikipedia: Query Optimization
- Use The Index, Luke: Explain Plan

Quick Revision Cheat Sheet

- **Parser:** Syntax check. **Optimizer:** Plan selection. **Executor:** Action.
- **Heuristic:** Rule-based (e.g., "Filter early").
- **CBO:** Stats-based (e.g., "Lowest I/O cost").
- **SARGable:** A query that can use an index (no functions on columns).
- **Explain:** Command used to see the chosen plan.
- **Join Order:** Smaller table first in Hash Joins.
- **Stale Stats:** The #1 reason for a "good index" being ignored.

END OF SUBTOPIC 13 — MOVING TO SUBTOPIC 14

Subtopic 14: Database Security (10 MCQs)

Quick Notes

- **Principal of Least Privilege (PoLP):** A user should be granted only the minimum level of access (privileges) necessary to perform their specific job functions.

- **Discretionary Access Control (DAC):** The owner of an object (e.g., a table) has the authority to grant or revoke access to that object to other users (using `GRANT` and `REVOKE`).
- **Role-Based Access Control (RBAC):** Privileges are assigned to roles (e.g., 'Analyst', 'Admin'), and users are assigned to those roles. This simplifies management.
- **SQL Injection (SQLi):** A vulnerability where an attacker "injects" malicious SQL code into a query via user input. Prevented primarily by **Prepared Statements** (Parameterized Queries).
- **Encryption:**
  - *At-Rest:* Protecting data on the physical disk (TDE - Transparent Data Encryption).
  - *In-Transit:* Protecting data as it travels over the network (SSL/TLS).
- **Database Auditing:** The process of tracking and logging database activities (who accessed what data and when).

Common Traps & Confusions

- **GRANT ALL:** A common security mistake in development. Never use `GRANT ALL` in production; it violates PoLP and provides a massive attack surface if a service account is compromised.
- **Obscurity vs. Security:** Changing a database port from 3306 to 9000 is "Security by Obscurity." It might slow down a script kiddie but won't stop a determined attacker. Real security relies on robust authentication and authorization.
- **SQLi in Stored Procedures:** Many believe stored procedures are inherently safe from SQLi. This is false; if the procedure uses dynamic SQL (string concatenation) inside, it is still vulnerable.
- **Revoking "Public":** In some systems (like SQL Server), every user is a member of the `public` role. Revoking permissions from a specific user but leaving them on `public` might mean the user still has access.

MCQs

Q1. (Scenario-Based) A newly hired intern needs to generate a report on monthly sales. The intern should be able to read the `Orders` and `Products` tables but must not be able to modify them or see the `Salaries` table. Which security principle is the DBA applying by granting only `SELECT` on those two specific tables?

A) Separation of Duties

B) Mandatory Access Control (MAC)

C) Principle of Least Privilege (PoLP)

D) Defense in Depth

> PoLP states that a user should only have the exact permissions required for their task. Granting `SELECT` (read-only) on only the relevant tables follows this principle strictly.

Q2. (Code-Tracing) Trace the result of the following DCL commands. Can 'User_A' delete records from the `Logs` table at Time T4?

sql-auth-monitor

```
T1: CREATE ROLE 'DataCleanup';
T2: GRANT SELECT, DELETE ON public.Logs TO 'DataCleanup';
T3: GRANT 'DataCleanup' TO 'User_A';
T4: -- User_A attempts: DELETE FROM Logs WHERE id = 101;
```

A) Yes, because User_A inherited the DELETE privilege from the 'DataCleanup' role.

B) No, because User_A was not granted the privilege directly.

C) No, because roles cannot hold DML privileges.

D) Yes, but only if User_A is also the owner of the table.

> This is the core of Role-Based Access Control (RBAC). Privileges granted to a role are automatically inherited by any user assigned to that role, making permission management much more scalable.

Q3. (Scenario-Based) A web application takes a `username` from a login form and builds a query like this:

`"SELECT * FROM Users WHERE username = '" + userInput + "';"`

An attacker enters `' OR '1'='1` into the field. What is the most effective architectural way to prevent this **SQL Injection** attack?

A) Using a firewall to block all traffic to the database port.

B) Encrypting the `username` column in the database.

C) Using Prepared Statements with Parameterized Queries.

D) Writing a JavaScript function to remove single quotes from the input.

> Prepared statements pre-compile the SQL structure. The `userInput` is then sent as a literal parameter, not as executable code. The database engine will look for a user literally named `' OR '1'='1` rather than executing the logic.

Q4. (Code-Tracing) A medical database contains sensitive patient data. The DBA creates the following object. What security benefit does this provide?

postgres - view_security

```
CREATE VIEW Public_Patient_Stats AS
SELECT city, COUNT(*) as patient_count
FROM Patients
GROUP BY city;

GRANT SELECT ON Public_Patient_Stats TO 'researcher_role';
REVOKE SELECT ON Patients FROM 'researcher_role';
```

A) It makes the query run faster using materialization.

B) It prevents researchers from seeing any data at all.

C) It provides "Vertical and Horizontal Filtering" to expose aggregate data while hiding individual sensitive records (PII).

D) It automatically encrypts the `city` column.

> Views are a powerful security tool. By granting access to the view but not the underlying table, the DBA can strictly control which columns and rows (or aggregates) a user can see, effectively masking Personally Identifiable Information (PII).

Q5. (Conceptual) In a highly secure government database, access is granted based on the sensitivity label of the data (e.g., Top Secret) and the clearance level of the user. The user cannot pass their access rights to others. This is known as:

A) Discretionary Access Control (DAC)

B) Mandatory Access Control (MAC)

C) Role-Based Access Control (RBAC)

D) Attribute-Based Access Control (ABAC)

> In MAC, access is centrally controlled by the system based on security labels. Unlike DAC (where a table owner can share their table), MAC does not allow users to determine who else can access the data.

Q6. (Scenario-Based) A company needs to show customer support agents the last 4 digits of a credit card number (e.g., `XXXX-XXXX-XXXX-1234`) but hide the rest. Which database security technique is specifically used for this?

A) Transparent Data Encryption (TDE)

B) Dynamic Data Masking (DDM)

C) Symmetric Encryption

D) Database Auditing

> Data Masking hides sensitive data in the result set of a query without changing the actual data on the disk. It is ideal for call centers or support environments where only partial info is needed.

Q7. (Comparison) What is the primary architectural difference between **Data-at-Rest Encryption** and **Data-in-Transit Encryption**?

A) At-Rest uses RSA; In-Transit uses AES.

B) At-Rest protects against hackers; In-Transit protects against employees.

C) At-Rest protects physical files on the disk (e.g., from stolen hard drives); In-Transit protects data as it moves over the network (e.g., from packet sniffing).

D) At-Rest is mandatory for SQL; In-Transit is only for NoSQL.

> At-Rest encryption (like TDE) ensures that if a backup tape or hard drive is stolen, the data cannot be read. In-Transit encryption (like TLS/SSL) ensures that an attacker on the same network cannot "see" the SQL queries or results being exchanged between the app and the DB.

Q8. (Code-Tracing) An auditor reviews the following system log. Which security vulnerability is being attempted here?

db-audit-log

```
[2023-10-01 12:00] User: 'guest_acc'
[2023-10-01 12:00] Query: SELECT * FROM users WHERE id = 1; DROP TABLE
```

A) Buffer Overflow

B) Cross-Site Scripting (XSS)

C) SQL Injection (Piggybacked Query)

D) Denial of Service (DoS)

> The attacker is using a semicolon (`;`) to terminate the legitimate query and "piggyback" a second, malicious command (`DROP TABLE`). The `--` at the end is used to comment out the rest of the original query so it doesn't throw a syntax error.

Q9. (Scenario-Based) A financial organization requires that the person who **writes** the software to update account balances must not be the same person who **administers** the production database. This is a classic example of:

A) Least Privilege

B) Separation of Duties (SoD)

C) Multi-Factor Authentication

D) Non-Repudiation

> SoD is a security principle where more than one person is required to complete a critical task, preventing any single individual from having enough power to commit and conceal fraud.

Q10. (Conceptual) Which of the following is the primary goal of **Database Auditing**?

A) To encrypt sensitive columns automatically.

B) To speed up queries by logging frequently used parameters.

C) To maintain a chronological record of database activities to ensure accountability and detect unauthorized access.

D) To prevent SQL injection by filtering input.

> Auditing doesn't prevent an attack; it records it. It is essential for compliance (like HIPAA or GDPR) and for forensic analysis after a security incident to determine what was stolen or changed.

Deep Dive: Parameterized Queries and the Query Plan

To understand why Prepared Statements (Parameterized Queries) are the ultimate defense against SQL Injection, look at the **Query Optimizer**:

1. **Without Parameterization:** The query `SELECT * FROM users WHERE name = 'John'` is parsed and a plan is created. If the next query is `... WHERE name = 'Admin'`, the engine might have to parse it all over again because the string is different. If the input is `' OR 1=1`, the parser sees `OR` as a **keyword** and changes the logic of the query tree.
2. **With Parameterization:** You send `SELECT * FROM users WHERE name = ?`. The engine parses this and creates a fixed "Execution Plan" where the `?` is strictly defined as a **data literal**. When you later send the value `' OR 1=1`, the engine does not re-parse the query. It simply looks for a name that matches that exact string. The `OR` is treated as a character, not a logical operator.

This not only provides security but also **Performance**, as the database can reuse the same execution plan for thousands of different inputs.

Stable Web Resources

- OWASP: SQL Injection Guide
- Wikipedia: Database Security

Quick Revision Cheat Sheet

- **Least Privilege:** Only the permissions needed, nothing more.
- **GRANT/REVOKE:** DCL commands for access control.
- **SQLi Defense:** Prepared Statements / Parameterized Queries.
- **Encryption At-Rest:** TDE (protects files).
- **Encryption In-Transit:** TLS/SSL (protects network).
- **RBAC:** Roles → Privileges; Users → Roles.
- **Auditing:** Who, what, when, where (Accountability).

END OF SUBTOPIC 14 — MOVING TO SUBTOPIC 15

Subtopic 15: Distributed Databases (10 MCQs)

Quick Notes

- **Distributed Database (DDB):** A single logical database that is physically spread across multiple computers (nodes) in a network.
- **Fragmentation:**
  - *Horizontal:* Splitting a table by rows (e.g., North American customers in one node, European in another).
  - *Vertical:* Splitting a table by columns (e.g., Name/ID in one node, Credit Card info in another).
- **Replication:** Storing copies of data on different nodes to improve availability and read performance.
- **Distributed Transparency:** The user should interact with the database as if it were a single, local system (Location, Fragmentation, and Replication Transparency).
- **CAP Theorem:** In a distributed system, you can only guarantee two out of three: Consistency, Availability, and Partition Tolerance.
- **Two-Phase Commit (2PC):** A protocol to ensure atomicity in distributed transactions (Prepare Phase and Commit Phase).

Common Traps & Confusions

- **Distributed vs. Centralized with Remote Access:** Simply having a database on a cloud server that users access remotely is NOT a distributed database. A DDB must have data physically partitioned across multiple autonomous nodes.
- **CAP Theorem "Pick 2":** Students often forget that in a network-based system, Partition Tolerance (P) is usually non-negotiable. Therefore, the choice is effectively between Consistency (C) and Availability (A).

- **Replication is NOT Backup:** Replication is for high availability. If you accidentally `DROP TABLE` on a replicated database, the command is replicated instantly, and you lose data on all nodes. A backup is a point-in-time copy.

MCQs

Q1. (Scenario-Based) A global retail company stores its `Inventory` table across three servers: Tokyo, London, and New York. The Tokyo server only contains rows where `region = 'Asia'`. What specific type of distributed database design is this?

A) Vertical Fragmentation

B) Horizontal Fragmentation (Sharding)

C) Full Replication

D) Heterogeneous Distribution

> Horizontal fragmentation (also called sharding) involves dividing a table into subsets of rows based on a specific attribute (like region). This keeps data geographically close to the users who access it most frequently.

Q2. (Conceptual) According to the **CAP Theorem**, if a network partition occurs (nodes cannot communicate), a database that chooses to remain **Available** (A) must sacrifice which of the following?

A) Partition Tolerance (P)

B) Data Durability (D)

C) Strong Consistency (C)

D) Performance

> If nodes cannot talk to each other (Partition), and you allow users to keep writing data to any available node (Availability), those nodes will eventually hold different data. Therefore, you can no longer guarantee that every user sees the exact same "Consistent" state.

Q3. (Code-Tracing) A user executes the following query in a distributed environment. If the system possesses **Location Transparency**, what is true about the user's knowledge?

distributed-shell

```
SELECT name, balance
FROM Global_Accounts
```

```
WHERE account_id = 9001;
```

A) The user must specify the IP address of the server holding account 9001.

B) The user must know whether the table is fragmented or replicated.

C) The user does not need to know where the data is physically stored; the DBMS handles the routing.

D) The query will only work if the data is stored on the local machine.

> Location Transparency is a property of a DDBMS where the user can query any table by its logical name without needing to know the physical node or site where the data resides.

Q4. (Comparison) What is the primary disadvantage of **Synchronous (Eager) Replication** compared to Asynchronous (Lazy) Replication?

A) It leads to data inconsistency between nodes.

B) It significantly increases transaction latency because the "Commit" must wait for all replicas to confirm the write.

C) It requires more disk space than asynchronous replication.

D) It cannot be used for read-heavy workloads.

> In synchronous replication, a transaction is not considered "Committed" until all replicas have written the data. This ensures high consistency but means the user must wait for multiple network round-trips, slowing down every write operation.

Q5. (Scenario-Based) A bank is performing a distributed transaction to transfer money between a branch in Karachi and a branch in Islamabad. The **Two-Phase Commit (2PC)** protocol is used. If the Karachi node sends a "REDO" (Ready to Commit) message but the Islamabad node sends an "ABORT" message during the first phase, what happens?

A) Karachi commits, and Islamabad rolls back.

B) The coordinator sends a global ROLLBACK message to all nodes.

C) The transaction is suspended until Islamabad is ready.

D) The Islamabad node is ignored.

> The 2PC protocol follows an "All or Nothing" rule. If even one node votes to ABORT during the first phase (Prepare phase), the coordinator must instruct every node in the transaction to rollback to maintain global atomicity.

Q6. (Conceptual) In a **Heterogeneous** Distributed Database System, which of the following is true?

A) All nodes must run the exact same DBMS software (e.g., all MySQL).

B) All nodes must have the same hardware specifications.

C) Different nodes may run different DBMS software (e.g., some Oracle, some PostgreSQL) using a common interface.

D) Data is only stored in one central location.

> Heterogeneous systems integrate different types of database software across nodes. This is common during company mergers where different departments use different database technologies but need a unified query view.

Q7. (Comparison) Why would a database designer choose **Vertical Fragmentation** for a `Personnel` table?

A) To keep employees from different cities on different servers.

B) To separate frequently accessed non-sensitive data (Name, Dept) from rarely accessed sensitive data (SSN, Salary) for security and performance.

C) To increase the total number of rows the database can hold.

D) To avoid using Primary Keys.

> Vertical fragmentation splits a table by columns. By placing sensitive columns on a highly secure, restricted server and public columns on a faster, more accessible server, the organization improves both security and I/O efficiency for common queries.

Q8. (Logic/Calculation) A distributed query requires joining Table A (Node 1) and Table B (Node 2). To minimize network traffic, the DBMS sends only the unique Join-Keys from Table A to Node 2, filters Table B there, and sends back only the matching rows. What is this optimization technique called?

A) Full Join

B) Remote Join

C) Semi-Join

D) Distributed Scan

> The Semi-join optimization is a standard technique in distributed databases. It reduces network overhead by ensuring that only the relevant, matching rows are transferred across the wire, rather than sending the entire contents of a massive table.

Q9. (Scenario-Based) A social media application uses **Eventual Consistency** for "Likes" on a post. A user in London likes a post, but a user in Sydney doesn't see that like for another 2 seconds. Which property of a distributed system is being prioritized here?

A) Strong Consistency

B) High Availability and Low Latency

C) Data Integrity

D) Vertical Scalability

> Eventual consistency is a trade-off. By not forcing every node in the world to synchronize immediately (which would be slow), the system remains fast (Low Latency) and stays online even if some nodes are slow (Availability), with the promise that all nodes will "eventually" be the same.

Q10. (Conceptual) What is the primary role of the **Transaction Coordinator** in a distributed database?

A) To encrypt the data on each node.

B) To manage the execution of a single transaction across multiple nodes and ensure its global atomicity.

C) To act as the primary storage for all database logs.

D) To generate the HTML for the user dashboard.

> The coordinator is the "Master" node for a specific transaction. It initiates the 2PC protocol, collects votes from participating "Participant" nodes, and issues the final global decision (Commit or Abort).

Deep Dive: The Semi-Join Algorithm Mechanics

In distributed databases, the "Network Cost" is usually the bottleneck, often 100x slower than local disk I/O. Suppose we need to join `Employees` (Site 1, 10,000 rows) and `Departments` (Site 2, 50 rows) on `dept_id`. Instead of shipping 10,000 employee rows to Site 2, the DDBMS performs a **Semi-Join**:

1. Site 2 sends the list of its 50 `dept_ids` to Site 1. (Very small transfer).
2. Site 1 filters the `Employees` table locally, keeping only those who belong to those 50 departments.
3. Site 1 sends the filtered employees to Site 2 for the final join.

This drastically reduces the amount of data crossing the network, which is the primary goal of distributed query optimization.

Stable Web Resources

- Wikipedia: Distributed Database
- Wikipedia: CAP Theorem

Quick Revision Cheat Sheet

- **Horizontal Fragmentation:** Rows split by a condition (e.g., Region).
- **Vertical Fragmentation:** Columns split into different tables/nodes.
- **Replication:** Multiple copies for speed/safety.
- **CAP:** Pick 2 (Consistency, Availability, Partition Tolerance).
- **2PC:** Prepare Phase + Commit Phase (Ensures Distributed Atomicity).
- **Semi-Join:** Optimize by sending only keys/filtered rows over the network.
- **Autonomy:** Each node in a DDB can operate as a local database.

END OF SUBTOPIC 15 — MOVING TO SUBTOPIC 16

Subtopic 16: NoSQL & Modern Databases (10 MCQs)

Quick Notes

- **NoSQL (Not Only SQL):** A category of database management systems designed for distributed data stores where very large scale, high-velocity data, and flexible schemas are required.

- **Types of NoSQL Databases:**
  - *Document Stores:* Data stored as JSON/BSON documents (e.g., MongoDB, CouchDB).
  - *Key-Value Stores:* Simple hash-table based storage (e.g., Redis, DynamoDB).
  - *Column-Family Stores:* Optimized for reading large amounts of data across rows (e.g., Cassandra, HBase).
  - *Graph Databases:* Optimized for traversing relationships (e.g., Neo4j).
- **BASE Properties:** Basically Available, Soft-state, Eventual consistency. (Contrast with ACID).
- **Scaling:**
  - *Vertical (Scale-up):* Adding more CPU/RAM to a single server.
  - *Horizontal (Scale-out):* Adding more servers to a cluster (Sharding). NoSQL is built for this.
- **Schemaless:** Data can be inserted without a predefined structure, allowing for rapid evolution of data models.

Common Traps & Confusions

- **"NoSQL means no SQL":** Many NoSQL databases actually support SQL-like query languages (e.g., CQL for Cassandra or N1QL for Couchbase). The "No" stands for "Not Only".
- **Consistency Misconception:** NoSQL is not "Always Inconsistent." Many NoSQL systems allow you to tune consistency (e.g., Read-at-Quorum) to act like an ACID database when needed.
- **Joins:** NoSQL generally does not support server-side Joins. If you need to join data, you either denormalize the data (duplicate it) or perform the join in the application code.
- **Vector Databases:** A modern trend for AI. They store data as high-dimensional vectors (embeddings) to allow for "Similarity Search" rather than exact keyword matches.

MCQs

Q1. (Scenario-Based) A startup is building a real-time analytics dashboard for a global IoT network. Sensors send millions of small updates every second. The schema changes

frequently as new sensor types are added. Which database type is most appropriate for this high-velocity, flexible-schema workload?

A) A Relational Database (PostgreSQL)

B) A Document Store (MongoDB) or Column-Family Store (Cassandra)

C) A Hierarchical Database (IMS)

D) An Excel Spreadsheet

> NoSQL databases like MongoDB or Cassandra are designed for "Horizontal Scaling" (handling massive velocity by adding nodes) and "Schema Flexibility" (handling changing data structures without migrations), making them ideal for IoT and real-time big data.

Q2. (Code-Tracing) Analyze the following operation from a **Document-oriented database**. What is the primary difference between this and a traditional SQL `INSERT`?

mongo-shell

```
db.products.insertOne({
  "item": "Laptop",
  "specs": { "cpu": "i7", "ram": "16GB" },
  "tags": ["electronics", "sale"]
});
```

A) It requires a pre-defined table structure.

B) It allows for nested objects and arrays within a single record, reducing the need for Joins.

C) It automatically encrypts the `tags` array.

D) It converts the data into a CSV format before storage.

> Document databases allow "Nested Data Models." In SQL, `specs` and `tags` would likely require separate tables and Joins. In NoSQL, related data is co-located in one document for faster retrieval.

Q3. (Conceptual) The **BASE** consistency model used by many NoSQL systems prioritizes "Availability" and "Partition Tolerance". What does the **"Soft-state"** property imply?

A) The data is stored in volatile RAM only.

B) The database automatically deletes old records to save space.

C) The state of the system may change over time, even without input, as different

nodes synchronize to reach consistency.

D) The database can only store strings and booleans.

> Soft-state means the system is "not stable" at all times. Due to "Eventual Consistency," the data on Node A might differ from Node B for a short window until the background synchronization processes align them.

Q4. (Comparison) In a relational database, you "Scale Up" (Vertical Scaling). In NoSQL, you "Scale Out" (Horizontal Scaling). What is the main architectural challenge of **Scaling Out**?

A) It is more expensive to buy many small servers than one giant server.

B) It makes the SQL syntax more complex.

C) It requires complex data partitioning (Sharding) and dealing with the CAP theorem trade-offs.

D) It limits the database to a maximum of 1,000 rows.

> Scaling out involves distributing data across a cluster. This introduces the "Network Partition" problem. Deciding how to split data (Sharding) so that queries don't have to hit every single server is the primary challenge of horizontal scaling.

Q5. (Scenario-Based) A social media company needs to implement a "Friend Recommendation" engine. The engine needs to traverse relationships like "Friends of Friends who also like the same Page." Which NoSQL model is mathematically optimized for this type of deep relationship traversal?

A) Key-Value Store

B) Document Store

C) Graph Database

D) Wide-Column Store

> Graph databases (like Neo4j) treat relationships as "first-class citizens." Instead of performing multiple index-lookups or joins, they use "pointer chasing" to traverse links between nodes in constant time, making them exponentially faster for social network analysis.

Q6. (Code-Tracing) A developer is using **Redis** to store session data. Which of the following commands demonstrates the "Key-Value" nature of this database?

```
redis-cli
```

```
T1: SET session:user:101 "{'id': 101, 'name': 'Alice'}"
T2: EXPIRE session:user:101 3600
T3: GET session:user:101
```

A) T3, because it retrieves a specific blob of data using a unique string identifier.

B) T2, because it manages the hardware memory.

C) T1, because it defines a table schema for users.

D) All of the above are relational commands.

> Key-Value stores act like a massive distributed dictionary/hash-map. You provide a Key (`session:user:101`) and get back a Value (the JSON string). There is no "internal" structure visible to the database for that value; it is just a blob.

Q7. (Comparison) Why is **Denormalization** considered a standard "Best Practice" in NoSQL design, whereas it is avoided in Relational (SQL) design?

A) Because NoSQL databases have infinite disk space.

B) Because NoSQL does not support Primary Keys.

C) Because NoSQL does not support Joins; duplicating data into one document allows for single-request reads.

D) Because denormalization makes data more consistent.

> In SQL, we normalize to save space and prevent anomalies. In NoSQL, "Disk is cheap, but Joins are expensive." By duplicating data (e.g., putting the Customer Name inside the Order document), we ensure that the entire order can be read in one operation without a network jump.

Q8. (Conceptual) What is the primary use-case for a **Column-Family Database** (like Cassandra or HBase)?

A) Small, simple mobile app backends.

B) Storing complex 3D image assets.

C) Analytical queries on massive datasets where the engine only needs to read a few specific columns across billions of rows.

D) Creating simple text-based blogs.

> Wide-column stores group data by column on the disk. If you want to calculate the "Average Price" across 1 billion rows, the engine only reads the `price` column from the disk, skipping the other attributes. This results in massive I/O savings for analytics.

Q9. (Scenario-Based) A modern AI company needs to build a search engine that finds "Images similar to this one." They use a **Vector Database**. How does this search work technically?

A) By matching the file name of the images.

B) By performing a `LIKE '%image%'` query on the metadata.

C) By calculating the "Cosine Similarity" (distance) between the mathematical vector of the input and the vectors in the database.

D) By converting images into SQL tables.

> Vector databases (like Pinecone or Milvus) store embeddings (arrays of numbers representing features). Searching involves finding vectors that are "closest" to the input vector in high-dimensional space, enabling "Semantic Search" or similarity matching.

Q10. (Conceptual) Which property of **NewSQL** databases (like CockroachDB or Google Spanner) distinguishes them from traditional NoSQL?

A) They do not support SQL.

B) They cannot scale horizontally.

C) They provide the horizontal scalability of NoSQL while maintaining full ACID compliance and SQL support.

D) They only run on specialized hardware.

> NewSQL is a modern class of RDBMS that seeks to provide the "Scale-out" benefits of NoSQL without sacrificing the "Strong Consistency" and ACID transactions that enterprise financial systems require.

Deep Dive: Polyglot Persistence

In modern architecture, the question is no longer "SQL or NoSQL?". Instead, architects use **Polyglot Persistence**—using multiple different database technologies within a single application, each for what it does best:

- **Relational (SQL):** For structured data with complex relationships and strict consistency needs (e.g., Users, Financial Transactions).
- **Redis (Key-Value):** For ultra-fast caching of session data and leaderboard scores in RAM.
- **ElasticSearch (Document/Search):** For full-text search across product catalogs.
- **Neo4j (Graph):** For the social connection engine or fraud detection patterns.
- **Prometheus (Time-Series):** For monitoring server metrics and logs over time.

This approach ensures that every microservice uses the data model that optimizes its specific performance and scaling requirements.

Stable Web Resources

- Wikipedia: NoSQL
- AWS: What is NoSQL?

Quick Revision Cheat Sheet

- **Document:** JSON-like, flexible, co-located data. Best for Web/Mobile.
- **Key-Value:** Dictionary-style. Best for Caching/Sessions.
- **Graph:** Nodes/Edges. Best for Recommendations/Social.
- **Wide-Column:** Columnar storage. Best for Big Data/Analytics.
- **Horizontal Scaling:** Scaling by adding nodes (The NoSQL strength).
- **Eventual Consistency:** "It'll be consistent soon, but not necessarily right now."
- **Schemaless:** Insert data now, figure out the structure later.

END OF SUBTOPIC 16 — MOVING TO SUBTOPIC 17

Subtopic 17: Data Warehousing & Data Mining (Introductory) (15 MCQs)

Quick Notes

- **Data Warehouse (DW):** A central repository of integrated data from one or more disparate sources, used for reporting and data analysis.
- **OLTP vs. OLAP:**
  - *OLTP (Online Transactional Processing):* Day-to-day operations (Inserts/Updates). Normalized. Focuses on speed.
  - *OLAP (Online Analytical Processing):* Complex historical queries. Denormalized. Focuses on multidimensional analysis.
- **ETL Process:** Extract (get data), Transform (clean/format), Load (insert into DW).
- **DW Schemas:**
  - *Star Schema:* One central Fact table connected to multiple Dimension tables. Simplest and fastest for joins.
  - *Snowflake Schema:* A normalized Star Schema where dimension tables are broken down into further sub-dimensions.
- **Data Mining:** The process of discovering patterns in large data sets (KDD - Knowledge Discovery in Databases).
- **Mining Tasks:** Classification (predicting labels), Regression (predicting numbers), Clustering (grouping), Association (finding relationships).

Common Traps & Confusions

- **Data Warehouse vs. Database:** A database is for "running" the business (Current data). A warehouse is for "analyzing" the business (Historical data).
- **Fact vs. Dimension:** A *Fact* is a quantitative measurement (e.g., Sales Amount, Quantity). A *Dimension* is the context (e.g., Time, Location, Product).

- **Data Mart:** A subset of a Data Warehouse, usually oriented to a specific business line or team (e.g., a "Marketing Data Mart").
- **Correlation is not Causation:** A common data mining pitfall. Finding that "Ice cream sales and drowning incidents increase together" (Correlation) doesn't mean one causes the other (both are caused by Summer).

MCQs

Q1. (Comparison) A bank uses two systems. System A processes every ATM withdrawal and update instantly. System B generates a report every quarter showing spending trends across different age groups over the last 5 years. How are these systems categorized?

A) Both are OLTP systems.

B) Both are OLAP systems.

C) System A is OLTP; System B is OLAP.

D) System A is a Data Mart; System B is a Data Warehouse.

> OLTP (System A) handles operational, atomic transactions. OLAP (System B) handles complex, historical, and multi-dimensional analytical queries for decision-making.

Q2. (Code/Logic Tracing) Look at the following SQL structure. Which Data Warehouse schema is being implemented here?

dw-schema-design

```
-- Central Table: Sales_Fact (Product_ID, Date_ID, Store_ID, Amount)
-- Linked Tables:
--   Product_Dim (Product_ID, Name, Category)
--   Date_Dim (Date_ID, Year, Month, Day)
--   Store_Dim (Store_ID, City, Region)
```

A) Star Schema

B) Snowflake Schema

C) Relational 3NF Schema

D) Galaxy Schema

> This is a classic Star Schema. There is a single, central Fact table (Sales_Fact) surrounded by non-normalized, flat Dimension tables. Each dimension is directly connected to the fact table by a single join.

Q3. (Scenario-Based) An e-commerce company wants to predict whether a customer will "Churn" (stop using the service) based on their last 6 months of activity. Which data mining task is required?

A) Association Rule Mining

B) Clustering

C) Classification

D) Regression

> Classification is used to predict a discrete label or category (e.g., Churn vs. No-Churn, Spam vs. Not-Spam). Regression would be used if the company wanted to predict a continuous numerical value, like the "Probability" of churn.

Q4. (Conceptual) During the **ETL (Extract, Transform, Load)** process, in which phase does "Data Cleaning" and "Deduplication" primarily occur?

A) Extract

B) Transform

C) Load

D) Data Warehousing

> The Transformation phase is the most complex part of ETL. This is where data from different sources is mapped to a common format, cleaned of errors, and merged to remove duplicate records before being loaded into the Warehouse.

Q5. (Calculation/Logic) In a multi-dimensional **Data Cube** representing Sales, the user moves from looking at "Sales by Year" to "Sales by Quarter" to "Sales by Month". What is this OLAP operation called?

A) Roll-up

B) Drill-down

C) Slice

D) Dice

> Drill-down navigates from less-detailed data (Year) to more-detailed data (Quarter/Month). The reverse (moving from Month to Year) is called Roll-up.

Q6. (Scenario-Based) A supermarket analyzes millions of receipts and discovers that people who buy "Diapers" on Friday nights are also 70% likely to buy "Beer". Which data mining technique was used?

A) Regression Analysis

B) K-Means Clustering

C) Association Rule Mining (Market Basket Analysis)

D) Classification

> Association Rule Mining finds interesting correlations and co-occurrences between items in a dataset. Market Basket Analysis is the most common application of this technique.

Q7. (Comparison) What is the primary architectural difference between a **Data Warehouse** and a **Data Mart**?

A) A Warehouse is for NoSQL data; a Mart is for SQL data.

B) A Warehouse is enterprise-wide and integrates all data; a Mart is a smaller subset focused on a specific department (e.g., Finance).

C) A Warehouse is for real-time data; a Mart is for historical data.

D) A Warehouse is normalized; a Mart is always denormalized.

> A Data Warehouse is the "Single Source of Truth" for the entire organization. A Data Mart is a decentralized slice of that data tailored for the specific needs of a single business unit, making it easier and faster for them to query.

Q8. (Conceptual) In Data Mining, the **KDD (Knowledge Discovery in Databases)** process defines several steps. Which of the following represents the correct sequence?

A) Mining → Cleaning → Transformation → Selection

B) Selection → Cleaning → Transformation → Mining → Interpretation

C) Interpretation → Mining → Selection → Loading

D) Loading → Cleaning → Mining → Cube Construction

> KDD starts with selecting the relevant data, cleaning/preprocessing it, transforming it into a mineable format, applying mining algorithms, and finally interpreting/evaluating the results.

Q9. (Logic/Schema) How does a **Snowflake Schema** differ from a Star Schema regarding **Normalization**?

A) It is less normalized than a Star Schema.

B) It is more normalized; dimension tables are broken down into smaller, related tables to reduce redundancy.

C) It does not use Fact tables.

D) It is only used for unstructured data.

> A Snowflake Schema normalizes the dimension tables (e.g., splitting a `Product` dimension into `Product` and `Category` tables). This saves space but results in more complex queries with more JOINs compared to a Star Schema.

Q10. (Scenario-Based) A researcher has a dataset of 1 million anonymous medical records. They use an algorithm to group patients into 5 distinct groups based on similar health metrics without knowing any pre-defined labels. This is an example of:

A) Supervised Learning (Classification)

B) Unsupervised Learning (Clustering)

C) Association Rule Mining

D) Regression

> Clustering is an "Unsupervised" task because the algorithm identifies natural groupings in the data without being told what those groups are (no training labels).

Q11. (Code/Logic Tracing) In a data mining context, what does **Confidence** represent in an association rule `{A} → {B}`?

association-metrics

```
-- Rule: {Bread} → {Milk}
-- Transaction Count: 100
-- Transactions with Bread: 40
-- Transactions with both Bread and Milk: 30
```

A) 30 / 100 (30%)

B) 30 / 40 (75%)

C) 40 / 100 (40%)

D) 10 / 40 (25%)

> Confidence is the probability that item B is purchased given that item A is purchased. Formula: `Count(A and B) / Count(A)`. Here: `30 / 40 = 0.75`. (30/100 would be the "Support" of the rule).

Q12. (Conceptual) What is a **"Factless Fact Table"** in a Data Warehouse?

A) A table that contains only historical metadata.

B) A fact table that contains only foreign keys to dimension tables, used to record the occurrence of an event (e.g., student attendance).

C) A dimension table that has been accidentally deleted.

D) A table used only for backup purposes.

> A Factless Fact table doesn't have numerical measures (like 'Amount'). It is used to track events, like "Student X attended Class Y on Date Z." The "measurement" is simply the count of those occurrences.

Q13. (Scenario-Based) A real estate website wants to estimate the selling price of a house based on its square footage, number of bedrooms, and location. Which data mining technique is appropriate?

A) Classification

B) Regression

C) Clustering

D) Association

> Regression is used to predict continuous, numerical outcomes (like a house price or temperature) based on input variables. Classification would be used if the goal were to predict "Cheap" vs "Expensive" labels.

Q14. (Comparison) What is the main reason an organization would choose **Cloud Data Warehousing** (e.g., Snowflake, BigQuery) over a traditional On-Premise Data Warehouse?

A) Cloud warehouses are more normalized.

B) Cloud warehouses do not require SQL.

C) Independent scaling of Storage and Compute, allowing for massive performance boosts for seasonal queries without buying permanent hardware.

D) Cloud warehouses are safer from network partitions.

> Modern cloud warehouses separate the "Disk" (Storage) from the "CPU" (Compute). If a company needs to run a giant end-of-year report, they can spin up 100 virtual CPUs for one hour and then turn them off, which is impossible with on-premise hardware.

Q15. (Conceptual) In OLAP operations, what does **"Slicing"** the data cube involve?

A) Combining two dimensions into one.

B) Selecting a single dimension to create a two-dimensional sub-cube (e.g., looking at Sales for only '2023').

C) Deleting a dimension from the cube.

D) Normalizing the cube into 3NF.

> A "Slice" picks one specific value of one dimension (e.g., Year = 2023) to see the rest of the data. A "Dice" picks a sub-range of multiple dimensions (e.g., Year 2023 AND City = Karachi).

Deep Dive: The Data Warehouse Bus Architecture

To ensure that different Data Marts (Finance, Sales, HR) can talk to each other, Ralph Kimball (a pioneer of DW) introduced the **Bus Architecture** based on **Conformed Dimensions**:

- **The Problem:** If the Finance team defines "Date" differently than the Sales team, their reports will never match.
- **The Solution:** The organization builds one single `Date_Dimension` and one single `Customer_Dimension` and shares them across every single Fact table in the enterprise.
- **Benefit:** This allows "Drill-across" queries, where you can join the Sales_Fact and the Finance_Fact tables because they both reference the exact same `Date_ID`. This creates a unified "Single Source of Truth."

Stable Web Resources

- Wikipedia: Data Warehouse
- Wikipedia: Data Mining

Quick Revision Cheat Sheet

- **OLTP:** Current, Normalized, Fast writes. **OLAP:** Historical, Denormalized, Fast reads.
- **Star Schema:** Center = Fact (Numbers). Points = Dimensions (Context).
- **Snowflake:** Normalized dimensions (more JOINs).
- **Classification:** Predict Label. **Regression:** Predict Number. **Clustering:** Group patterns.
- **Support:** How often item A appears. **Confidence:** Likelihood of B given A.
- **Roll-up:** Summarize. **Drill-down:** Detail.